

Aggregating Preferences for AI Alignment

Submitted for the Master of Science,
London School of Economics, University of London

Candidate Number: 42078

Department of Statistics 2024–25
London School of Economics and Political Science (LSE)

Acknowledgements

The submission of this dissertation marks the conclusion of my studies at the London School of Economics and Political Science. I greatly appreciate the experiences I have gained and the individuals I have met at this campus, which will remain among the most valuable memories of my life.

First and foremost, I would like to express my deepest gratitude to my supervisor, Professor Milan Vojnović, for his exceptional guidance, insightful feedback, and continuous support throughout the course of this research. His profound expertise and patience have been invaluable not only in guiding the direction of this work, but also in boosting my growth. Moreover, he has inspired me to stay curious, explore new ideas, and aim for contributions that benefit society.

Meanwhile, I am sincerely grateful to the Department of Statistics for providing essential computational resources and creating an intellectually stimulating academic environment, both of which were significant to the completion of this dissertation. My thankfulness extends to my teachers and fellow students, whose support, camaraderie, and shared enthusiasm have greatly enriched this academic pursuits.

Above all, I am profoundly grateful to my family, especially my grandfather, whose firm belief in my abilities and unconditional support have been a constant source of inspiration throughout every stage of my academic journey.

Finally, I acknowledge my own perseverance—for navigating uncertainty and steadily moving forward.

"We are star stuff, which has taken its destiny into its own hands." — Carl Sagan, Cosmos

Summary

Aligning large language models (LLMs) with human preferences remains a central challenge in natural language generation, particularly for open-ended tasks where correctness is subjective and evaluation criteria are ambiguous. Existing Reinforcement Learning (RL) approaches, such as Proximal Policy Optimization (PPO), focus on optimizing only the token-level advantage and require a large amount of computational power. Building on this, Group Relative Policy Optimization (GRPO) by DeepSeek generates multiple completions per prompt and extends its focus from the token level to the sequence level. However, it primarily optimizes for average performance during training and neglects to incorporate inference-time selection strategies, thereby causing a misalignment between the training and inference stages.

This work introduces *Marginal Utility-Based Policy Optimization* (MUBPO), an original RL fine-tuning framework that extends GRPO by integrating inference-time selection directly into the training objective. MUBPO leverages marginal utility theory to quantify the incremental contribution of each completion within a candidate set, with the marginal utility computable via a variety of utility functions. Among these, the maximum-utility formulation is particularly recommended for best-of-N sampling.

Experiments on the `TL;DR` Reddit Summarization dataset, using the `Qwen/Qwen2-0.5B-Instruct` model with the `Gemini-2.0-Flash` reward model, indicate that MUBPO consistently outperforms GRPO in fine-tuning, achieving higher task scores (approximately +1%) while maintaining lower Kullback–Leibler (KL) divergence (approximately -4) per sequence on average. Furthermore, increasing the number of candidate completions from four to eight yields an additional performance gain (approximately +3%), highlighting the benefits of richer completion sets during both training and inference. Collectively, these findings demonstrate that aligning training and inference objectives via marginal utility optimization improves both alignment fidelity and inference-time performance.

Additionally, MUBPO extends beyond being merely a RL algorithm tailored for LLMs. It also possesses the potential for broader applicability, encompassing domains such as autonomous decision-making, multi-agent system optimization, and adaptive resource allocation.

Keywords: Large Language Models, Reinforcement Learning, Reinforcement Learning from AI Feedback (RLAIF), Policy Optimization, Marginal Utility Theory, Post-Training, Fine-Tuning, Preference Modelling, AI Alignment.

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Research Motivation . . . 1
- 1.3 Contribution . . . 2
- 1.4 Outline . . . 2

2 Related Work **4**
- 2.1 RLHF . . . 4
- 2.2 Markov Decision Processes (MDP) . . . 5
- 2.3 Policy Optimization . . . 5
  - 2.3.1 PPO . . . 7
  - 2.3.2 GRPO . . . 8
  - 2.3.3 Pairwise Comparison-Based Method . . . 9
- 2.4 LLM-as-a-Judge . . . 9
- 2.5 BoN Sampling and the Training-Inference Mismatch . . . 10
- 2.6 Marginal Utility . . . 11

3 Methodology **13**
- 3.1 MUBPO Framework . . . 13
- 3.2 Utility Function Specifications . . . 14
  - 3.2.1 Summation Utility . . . 14
  - 3.2.2 Maximum Utility . . . 15
  - 3.2.3 Win Rate Utility* . . . 15
- 3.3 Advantage Metric . . . 16
- 3.4 Optimization Framework . . . 17
- 3.5 Inference-Time Evaluation . . . 19

4 Experiments **20**
- 4.1 Dataset and Reward Model . . . 20
- 4.2 MUBPO Algorithm . . . 20
- 4.3 Experimental Settings . . . 22
- 4.4 Prompt Design . . . 23
- 4.5 Model Specifications . . . 24

5 Results and Interpretation **26**
- 5.1 Sanity Check: Sample Completions . . . 26

5.2 Training Stage: *standard* vs. *max_m3* 26
5.3 Inference Stage: *standard* vs. *max_m3* 30
5.4 Training Stage: *max_m3* vs. *max_m3_n8* 33
5.5 Inference Stage: *max_m3* vs. *max_m3_n8* 34

6 Conclusion **37**

Bibliography **39**

Appendix **45**
I Policy Gradient Derivation 45
II Decomposition of Sequence-Level KL Divergence 47

List of Figures

2.1 Distribution Shift for Best-of-*N* Sampling 12

4.1 Training Reward Curve Using ROUGE-L 20

5.1 Max Score vs. Step for Current Policy and Reference Policy (*standard* vs. *max_m3*) . 28
5.2 Maximum and Minimum Advantage vs Step (*standard* vs. *max_m3*) 29
5.3 KL Divergence vs Step (*standard* vs. *max_m3*) 30
5.4 Mean Score and Mean KL Divergence with 95% Confidence Intervals (*standard* vs. *max_m3*) 31
5.5 Score vs KL Divergence (*standard* vs. *max_m3*) 32
5.6 Max Score vs Step for Current Policy (*max_m3* vs. *max_m3_n8*) 33
5.7 Maximum and Minimum Advantage vs Step (*max_m3* vs. *max_m3_n8*) 34
5.8 KL Divergence vs Step (*max_m3* vs. *max_m3_n8*) 35
5.9 Mean Score and Mean KL Divergence with 95% Confidence Intervals (*max_m3* vs. *max_m3_n8*) 35

List of Tables

2.1 Definitions of State, Action, and Policy Probabilities in Autoregressive Token Generation 7

4.1 Model Configuration 22
4.2 Random Seed Allocation for Reproducibility 22
4.3 Hardware and Execution Settings 23
4.4 Optimizer Hyperparameters 23
4.5 Sampling and Generation Settings 23
4.6 Training and Evaluation Datasets 24
4.7 Prompt Designs for Summarization and Scoring Tasks 24

5.1 Model Comparison with Input Text and Generated Summaries 27
5.2 Performance Comparison Between *standard* and *max_m3* Models 30

Chapter 1 Introduction

1.1 Background

In recent years, the rapid evolution of large language models (LLMs) has driven profound transformations across a wide array of sectors. In healthcare, LLMs have been deployed to enhance the efficiency and accuracy of clinical documentation; in financial services, they have supported more rigorous and data-driven approaches to risk assessment; and in education, they have enabled the delivery of personalized learning pathways. Despite the diversity of these applications, the operational roles of LLMs can be broadly classified into two overarching categories. The first comprises tasks with objectively verifiable solutions, such as programming or mathematical problem-solving, which we refer to as having 'ground-truth value'. For such tasks, aligning model behaviour with human preferences is comparatively straightforward, as correctness can be unambiguously assessed. The second category encompasses open-ended generative tasks, in which output quality is inherently subjective and resistant to formalization within the traditional supervised learning paradigm. In these contexts, alignment with human preferences presents greater challenges and necessitates specialized methodological approaches.

Reinforcement Learning from Human Feedback (RLHF) (Christiano et al., 2017; Stiennon et al., 2020) has emerged as the dominant paradigm for aligning language models with human preferences. RLHF incorporates human preferences into the training loop via a reward model trained on comparison data. The standard RLHF pipeline involves three main stages (Ouyang et al., 2022a): (1) supervised fine-tuning of a pretrained model on instructional data, (2) using comparison data to train a reward model to approximates human preferences, and (3) policy optimization through Reinforcement Learning (RL) using algorithms such as Proximal Policy Optimization (PPO) (Schulman et al., 2017). RLHF has underpinned the development of aligned models such as ChatGPT (OpenAI et al., 2024) and Claude ("The Claude 3 Model Family: Opus, Sonnet, Haiku", 2024), and has become a foundational component of modern LLM alignment practices.

Concurrently, superior and more efficient alternative policy optimization algorithms, suitable for implementation in the third stage of RLHF, have been widely explored. One of the most notable developments is the recently proposed Group Relative Policy Optimization (GRPO) by DeepSeek (DeepSeek-AI et al., 2025). GRPO improves upon PPO by enhancing sample efficiency through the generation of multiple completions per prompt and the application of group-normalized advantage estimates. Additionally, developments in evaluation methods, such as the use of LLMs as evaluators ("LLM-as-a-Judge"), have also advanced models' ability to align with human preferences. Together, these advances reflect an ongoing effort to better integrate preference data, probabilistic modelling, and RL into a coherent framework for language model alignment.

1.2 Research Motivation

Although RLHF has been widely adopted and has demonstrated considerable effectiveness in aligning LLMs with human preferences, several foundational limitations persist in the third stage of the pipeline, chief among them the training–inference mismatch, a systematic divergence between model behaviour during optimization in training and during deployment in inference.

The training process typically optimizes the policy under one of two regimes: (1) single-sample generation, in which the model produces exactly one output per prompt (e.g., PPO); or (2) multi-sample generation, wherein multiple completions are produced per prompt (e.g., GRPO), with the objective of maximizing the expected average reward within the completion group. In contrast, at inference, LLMs are frequently evaluated using decoding strategies such as Best-of-N (BoN) sampling, wherein multiple candidate completions are generated and the highest-reward completion is selected. This discrepancy manifests in two principal forms of misalignment:

- **Distribution shift**: The training objective is optimized for average-case performance, whereas the inference procedure is designed to maximize best-case performance. This training–inference mismatch induces a distribution shift, which is considered a form of forward misalignment (J. Ji et al., 2025). A detailed discussion of this phenomenon is provided in Section 2.3.2 and Section 2.5.
- **Granularity misalignment in optimization**: Although rewards are assigned at the sequence level during training, optimization occurs at the token level, introducing a granularity mismatch in the optimization process. This issue will be further analyzed in Section 2.3.2.

Collectively, we argue that these two misalignment problems undermine the reliability, stability, and overall effectiveness of the alignment process, thereby constraining the robustness of the third stage of RLHF in practical scenarios.

1.3 Contribution

To address these misalignment challenges, we propose the *Marginal Utility-Based Policy Optimization* (MUBPO) framework, a novel, unified and extensible approach with the following contributions:

- **Unified generalization:** MUBPO subsumes GRPO as a special case and accommodates a broad family of utility functions, enabling direct adaptation to diverse inference decoding strategies.
- **Marginal utility for RL:** We are the first to introduce the marginal utility formulation for RL in LLMs, enabling the quantification of the incremental contribution of each completion within a set.
- **Alignment Improvement:** MUBPO optimizes for the best-performing completion within each candidate set, aligning the training objective precisely with the inference decoding strategy. This design mitigates distribution shift, supports sequence-level optimization, and eliminates granularity mismatches.
- **Practical instantiation via maximum-utility optimization:** For the BON decoding strategy, we instantiate MUBPO with a maximum-utility objective that targets the highest-quality response in each set. Extensive experiments demonstrate consistent improvements over the GRPO baseline in both task performance and KL divergence.
- **Broad applicability:** Beyond large language model fine-tuning, MUBPO is a RL algorithm applicable to broader domains such as autonomous decision-making, multi-agent coordination, and adaptive resource allocation.

1.4 Outline

The remainder of this dissertation is structured as follows. Chapter 2 reviews related literature on RLHF, advantage estimation methods, policy optimization algorithms, evaluation paradigms such as

the LLM-as-a-Judge framework, and the marginal utility theory from microeconomics. Chapter 3 details the proposed MUBPO methodology, including the formulation of utility functions and the corresponding optimization strategy. Chapter 4 describes the dataset, MUBPO algorithm, experimental setup, and model specifications used for empirical validation. Chapter 5 presents and interprets the results, comparing the performance of the proposed method with that of the baseline GRPO, and evaluating the effect of increasing the number of completions. Finally, Chapter 6 concludes with a summary of findings and suggests directions for future work.

Chapter 2 Related Work

In this chapter, we begin by outlining the overall RLHF framework and then narrow our focus to its third stage—policy optimization via RL. We review the theoretical foundations of RL and PPO as applied in this stage, and subsequently extend the discussion to GRPO, highlighting the misalignment issues inherent to GRPO. We then examine the feasibility of employing a LLM as a reward model and justify the transition from RLHF to RL from AI Feedback (RLAIF). Thereafter, we present the BoN decoding strategy, providing a detailed examination of the training–inference mismatch and the resulting distribution shift. The chapter concludes by introducing the concept of marginal utility, a classical concept from microeconomics, which serves as the theoretical foundation for our proposed MUBPO algorithm.

2.1 RLHF

RLHF (Christiano et al., 2017; Stiennon et al., 2020) is a training paradigm that combines supervised learning with RL in order to better align model outputs with human preferences. Originally explored in the context of RL for interactive environments such as games, RLHF has since become a central method for adapting large pretrained models to natural language processing applications. Prominent examples include text summarization (Böhm et al., 2019; Wu et al., 2021; Ziegler et al., 2020), dialogue generation (Hancock et al., 2019; Yi et al., 2019), and machine translation (Bahdanau et al., 2017; Kreutzer et al., 2018). In all of these cases, the aim is not merely to improve task performance in a narrow metric sense, but to align model behaviour better with human preferences, which may be subtle, multidimensional, and difficult to encode as fixed rules.

The conventional RLHF fine-tuning pipeline, as popularized by Ouyang et al. (2022a), comprises three key stages:

1. Supervised Fine-Tuning (SFT): A large pretrained language model is first fine-tuned on high-quality, human-curated datasets for the target task. It ensures that the model has a reasonable baseline performance and produces outputs in a desirable format. Importantly, SFT serves as a "warm start" for subsequent RL training, reducing the risk of unstable optimization or catastrophic forgetting.
2. Reward Modelling: A reward model is trained to approximate human preferences. This requires collecting datasets where humans compare multiple model outputs for the same input and indicate which one they prefer. The reward model is then optimized to assign higher scores to preferred outputs and subsequently acts as a judge at a later stage.
3. Policy Optimization: Using the reward model from stage two, the SFT model is further fine-tuned via RL. PPO algorithm (Schulman et al., 2017) is a common choice at this stage due to its balance of stability and efficiency (Ouyang et al., 2022a). Since its objective is to maximize the predicted reward while constraining the updated policy to remain close to the SFT policy, it helps preserve linguistic fluency and factuality.

Overall, this pipeline has proven effective for training LLMs that align more closely with human judgement and communication. However, several challenges remain, including reward model bias and the high cost of collecting high-quality human preference data. Recent work has explored an

alternative method, RLAIF (Lee et al., 2024), which uses an LLM as the reward model to mitigate these problems. Section 2.4 will further examine the feasibility of this method. In this dissertation, we focus on developing policy optimization methods for the third stage, often referred to in a broad sense as post-training.

2.2 Markov Decision Processes (MDP)

RL addresses the problem of learning to act in an environment to maximize long-term rewards (Sutton & Barto, 2018). The environment is commonly modelled as a MDP, denoted by the tuple

$$(\mathcal{S}, \mathcal{A}, P, r),$$

where:

- $\mathcal{S}$ is the state space;
- $\mathcal{A}$ is the action space;
- $P(s'|s, a)$ is the transition probability from state s to s' when taking action a;
- $r(s, a)$ is the reward function for taking action a in state s.

A policy $\pi_\theta(a|s)$, parameterized by θ, defines the probability of taking action $a \in \mathcal{A}$ in state $s \in \mathcal{S}$.

Overall, the goal, i.e., the objective, is to maximize the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right], \tag{2.1}$$

where $\tau = (s_0, a_0, s_1, a_1, \ldots)$ is a trajectory generated by following π_θ, $\gamma \in [0, 1)$ is the discount factor controlling the importance of future rewards, and the expectation $\mathbb{E}_{\tau\sim\pi_\theta}[\cdot]$ is taken over the distribution of trajectories induced by the policy.

2.3 Policy Optimization

Based on Equation 2.1, the gradient of the expected return $J(\theta)$ with respect to the policy parameters θ can be expressed as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\right], \tag{2.2}$$

where $G_t = \sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k})$ denotes the empirical return following time step t. The derivation from Equation 2.1 to Equation 2.3 is detailed in Appendix I. In some literature, such as Sutton and Barto (2018), the reward is defined as r_{t+k+1}. Note that it is essentially the same as $r(s_{t+k}, a_{t+k})$, reflecting the fact that it is received after taking action a_{t+k} in state s_{t+k}.

In this formulation:

- $\log \pi_\theta(a_t \mid s_t)$ is the log-likelihood of the action under the current policy,
- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ is the score function,
- G_t serves as a Monte Carlo estimate of the action-value function $Q^\pi(s_t, a_t) = \mathbb{E}_\pi[\, G_t \mid s_t, a_t \,]$.

Equation 2.3 forms the basis of the REINFORCE algorithm (Williams, 1992), a fundamental Monte Carlo policy gradient method. Additionally, we have the following property in stochastic gradient estimation states:

$$\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] = 0, \tag{2.3}$$

which permits the introduction of baseline functions without biasing the gradient estimate. Therefore, the REINFORCE algorithm can be extended further by applying this identity into Equation 2.3:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot (G_t - b(s_t))\right], \tag{2.4}$$

where $b(s_t)$ is any function of the state alone. The inclusion of a baseline reduces the variance of the estimator (Ng & updated by Tengyu Ma, 2022), improving both learning stability and sample efficiency.

A common and effective choice for the baseline is the state value function:

$$V^\pi(s_t) = \mathbb{E}_\pi\left[G_t \mid s_t\right], \tag{2.5}$$

which leads to the following form of the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot A^\pi(s_t, a_t)\right], \tag{2.6}$$

where the advantage function is defined as $A^\pi(s_t, a_t) = G_t - V^\pi(s_t)$.

This formulation is widely adopted in actor-critic architectures (Konda & Tsitsiklis, 1999), wherein the actor updates the policy based on advantage estimates provided by a learned critic (i.e., a value function).

An alternative approach is to estimate both $Q_\pi(s_t, a_t)$ and $V_\pi(s_t)$ directly, and then compute the advantage function as

$$\hat{A}_t = Q_\pi(s_t, a_t) - V_\pi(s_t). \tag{2.7}$$

Although this formulation is theoretically consistent with the definition of the advantage function and has been adopted in prior research (Z. Wang et al., 2017), its practical implementation poses significant challenges when employing function approximation, in which Q_π and V_π are represented by parameterized models such as neural networks or other statistical regression frameworks. In such cases, the accuracy of $\hat{A}_t$ depends on the precision of both value estimates. Even small independent estimation errors in Q_π and V_π can compound when taking their difference, potentially leading to high-variance or biased advantage estimates. Hence, this method is less robust in practice.

A prominent variant in the actor–critic setting (Equation 2.6), known as the Advantage Actor–Critic (A2C) method (Mnih et al., 2016), uses a one-step temporal-difference (TD) estimate of the advantage:

$$\hat{A}_t = \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{2.8}$$

which trades off some bias for significant variance reduction.

Instead of committing to a single n-step return, where n is the number of steps we look ahead before bootstrapping with the value function, Generalized Advantage Estimation (GAE) Schulman, Moritz, et al. (2015) blends all n-step returns using an exponentially decaying weight:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \tag{2.9}$$

where $\lambda \in [0, 1]$ controls the bias-variance spectrum. Note that when $\lambda = 0$, it recovers the one-step TD estimate, whereas $\lambda = 1$ corresponds to the Monte Carlo return. In recent years, GAE has become a

popular advantage estimation method, adopted by numerous algorithms such as PPO (Schulman et al., 2017).

2.3.1 PPO

PPO (Schulman et al., 2017) uses a clipped surrogate objective:

$$J^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_t\big)\Big], \tag{2.10}$$

with the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

Here:

- $r_t(\theta)$ measures how the new policy differs from the old policy;
- ϵ is a hyperparameter that limits the size of the update;
- the min operator enforces a conservative improvement;
- the clip function prevents $r_t(\theta)$ from moving outside $[1-\epsilon, 1+\epsilon]$;
- $\hat{A}_t$ follows Equation 2.9.

In the context of LLMs, PPO operates on token-level MDPs, where each symbol corresponds to the definitions listed in Table 2.1.

Symbol	Definition	Description
s_t	$(v_1, v_2, \ldots, v_t) \in \mathcal{S}$	State at step t, consisting of the sequence of generated tokens up to time t, where each $v_k \in \mathcal{V}$ represents a token from the vocabulary.
a_t	$v_{t+1} \in \mathcal{A}$	Action at step t, corresponding to the next token predicted by the model.
$\pi_\theta(a_t \mid s_t)$	$\mathbb{P}(a_t\|s_t)$	Probability of generating token a_t given state s_t under the current policy π_θ.
$\pi_{\theta_{\text{old}}}(a_t \mid s_t)$	$\mathbb{P}_{\text{old}}(a_t\|s_t)$	Probability of generating token a_t given the same state s_t under the previous policy $\pi_{\theta_{\text{old}}}$.

Table 2.1: Definitions of State, Action, and Policy Probabilities in Autoregressive Token Generation

Each state s_t corresponds to the portion of the token sequence generated so far. Each action a_t corresponds to the next token generated by the model. The probability ratio $r_t(\theta)$ measures how the new policy π_θ changes its probability of generating the same token a_t compared to the old policy $\pi_{\theta_{\text{old}}}$. The advantage $\hat{A}_t$ is then derived from the reward model in RLHF, indicating whether generating a_t in context s_t is better or worse than expected. The clipping prevents the model from making extreme probability updates for any token, stabilizing LLM fine-tuning. By penalizing large policy updates implicitly, PPO achieves a balance between stability and simplicity, outperforming earlier actor-critic methods including A2C (Mnih et al., 2016)) and approximating TRPO-like (Schulman, Levine, et al., 2015) performance without costly second-order computations.

However, since PPO using GAE to to compute $\hat{A}_{i,t}$ (shown in Equation 2.9), the practical challenge of PPO lies in its heavy reliance on the value model. In particular, because the algorithm's effectiveness

depends heavily on the accuracy of its value estimates, obtaining a reliable value model is challenging, especially for long responses and complex tasks.

2.3.2 GRPO

GRPO extends PPO by generating a group of completions per prompt and leveraging group-based baselines to reduce variance and improve sample efficiency. There are two main variants:

1. Token-level GRPO, introduced in DeepSeekMath (Shao et al., 2024), is a token-level approach similar to PPO, and
2. Sequence-level GRPO, proposed in DeepSeek-R1 (DeepSeek-AI et al., 2025), which optimizes the policy on complete output sequences.

Token-level GRPO. Similar to PPO, GRPO from DeepSeekMath (Shao et al., 2024) uses a token-level objective function:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim P(Q),\, \tau_i \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \frac{1}{G}\sum_{i=1}^{G}\frac{1}{T_i}\sum_{t=0}^{T_i-1}\Bigg[\min\Big(r_{i,t}\,\hat{A}_{i,t},\ \text{clip}\big(r_{i,t}, 1-\epsilon, 1+\epsilon\big)\hat{A}_{i,t}\Big) - \beta\, D_{\text{KL}}\big(\pi_\theta(\cdot \mid s_{i,t}, q)\,\big\|\,\pi_{\text{ref}}(\cdot \mid s_{i,t}, q)\big)\Bigg], \tag{2.11}$$

$$r_{i,t} = \frac{\pi_\theta(a_{i,t} \mid s_{i,t}, q)}{\pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t}, q)}.$$

where G denotes the number of responses sampled for each prompt, $q \sim P(Q)$ indicates that a question q is sampled from the distribution $P(Q)$, and T_i represents the length of the i-th trajectory.

Compared with Equation 2.10, the key distinction lies in the averaging factor G, which reflects that multiple responses are generated per prompt, and their contributions to the objective are aggregated through averaging. Additionally, the computation of the advantage $\hat{A}_{i,t}$ differs between PPO and GRPO. PPO employs GAE to compute $\hat{A}_{i,t}$, while GRPO derives the advantage by standardizing the rewards across the batch:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \mu_R}{\sigma_R}, \tag{2.12}$$

where μ_R and σ_R represent the empirical mean and standard deviation of the rewards R_i within the group respectively.

By estimating the baseline from group scores instead of relying on a value function, GRPO substantially reduces training resource requirements. Additionally, compared to previous advantage estimation methods that only subtract a baseline value, GRPO further divides by the standard deviation, thereby mitigating scale variability across groups and facilitating stable policy optimization.

However, this design exhibits several limitations. The most fundamental, also observed in PPO, is the misalignment between the token-level formulation of the optimization objective and the sequence-level nature of the reward signal. This discrepancy can lead to ineffective credit assignment, as all tokens within a completion share the same advantage value (see Equation 2.12), thereby diluting the learning signal and potentially hindering convergence. A similar concern was raised in GSPO (C. Zheng et al., 2025), motivating the development of sequence-level policy optimization.

Sequence-level GRPO. Following the release of the token-wise GRPO implementation in its DeepSeekMath model, DeepSeek subsequently introduced a more comprehensive model, DeepSeek-R1 (DeepSeek-AI et al., 2025), which abandons the token-level objective in favor of a sequence-level formulation. The sequence-level objective is defined as follows:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim P(Q),\,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(O|q)}\Bigg[\frac{1}{G}\sum_{i=1}^{G}\min\Big(r_i(\theta)A_i,\ \text{clip}\big(r_i(\theta),\,1-\epsilon,\,1+\epsilon\big)A_i\Big) - \beta\, D_{\text{KL}}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)\Bigg], \tag{2.13}$$

$$r_i(\theta) = \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}.$$

A comparison of Equation 2.13 with Equation 2.11 reveals that the objective function is now defined over entire sequences rather than individual tokens, thereby eliminating the misalignment between reward assignment and the optimization objective. Therefore, our proposed model, MUBPO, likewise adopts a sequence-level objective instead of a token-level formulation. However, note that another misalignment problem still exists: sequence-level GRPO seeks to maximize the average advantage across the group, which may be suboptimal during the inference stage. By contrast, MUBPO could attain the best performance for each group through an appropriate choice of marginal utility method, such as the maximum utility function.

2.3.3 Pairwise Comparison-Based Method

All of the policy optimization methods discussed earlier rely on absolute rewards, where numerical values assigned to state–action transitions are used to compute the advantage. In recent years, however, an alternative paradigm has gained attention: using pairwise comparisons as the reward signal. Direct Preference Optimization (DPO) (Rafailov et al., 2023) is a widely adopted method in this paradigm. It models the probability that one response is preferred over another and optimizes the policy by maximizing the probability of preferred responses over dispreferred ones, without relying on an explicit reward model. Building on this idea, Nash Learning from Human Feedback (NLHF) (Munos et al., 2024) was proposed, which bypasses reward models by directly learning a pairwise preference model. Policy optimization is then framed as finding a Nash equilibrium, yielding a policy that cannot be consistently outperformed. More recently, InfAlign (Balashankar et al., 2025) has been introduced, which instead focuses on maximizing the inference-time win rate as its alignment objective.

In this dissertation, we introduce a utility function designed for the pairwise-comparison paradigm in the methodology chapter (see Section 3.2.3), but do not adopt it in the later experiments due to time and resource constraints.

2.4 LLM-as-a-Judge

In tasks with clearly defined ground-truth answers, such as mathematics and programming, evaluation is comparatively straightforward. In these domains, exemplified by models such as DeepSeek-R1 (DeepSeek-AI et al., 2025), performance can be objectively quantified using metrics such as accuracy and pass@k. In contrast, open-ended generation tasks, which lack a single correct answer,

have traditionally been evaluated using reference-based metrics, including BLEU (Papineni et al., 2002), ROUGE (Lin, 2004), METEOR (Banerjee & Lavie, 2005), and CIDEr (Vedantam et al., 2015). Although these metrics have played a foundational role, they exhibit inherent limitations in capturing higher-order qualities such as fluency, coherence, and semantic alignment (Novikova et al., 2017).

To mitigate these shortcomings, researchers have proposed multi-dimensional evaluation criteria (Barrett & Narayanan, 2008; Liu et al., 2025) and multi-objective reward models (H. Wang et al., 2024; Zhou et al., 2024). However, significant challenges persist in ensuring their robustness and maintaining consistency across dimensions. More recently, the LLM-as-a-Judge paradigm has emerged, in which large language models, such as OpenAI's GPT-4 (OpenAI et al., 2024), Anthropic's Claude ("The Claude 3 Model Family: Opus, Sonnet, Haiku", 2024), and Google DeepMind's Gemini (Team et al., 2025), serve as evaluators capable of delivering nuanced, context-aware, and semantically informed assessments that surpass the capabilities of traditional metrics.

Recent studies have shown that LLM-based evaluators correlate well with human judgments. L. Zheng et al. (2023) introduced a GPT-based framework for evaluating outputs and found consistent alignment with human preferences under open-ended questions especiall. Provakar (2024) further demonstrated that LLMs, when used as scoring functions, outperform traditional evaluation metrics. Beyond improved accuracy, LLM-based evaluation is both scalable and cost-effective, significantly reducing the overhead associated with extensive human annotation.

Nevertheless, concerns remain. LLM evaluators may inherit model biases (Ye et al., 2025) and suffer from preference leakage (Li et al., 2025), making reproducibility challenging. In addition, hallucinations and factual inaccuracies within the models themselves can undermine evaluation reliability (Z. Ji et al., 2023). Despite these limitations, the LLM-as-a-Judge paradigm marks a significant shift in how we assess language generation models, offering a promising alternative to both conventional metrics and human annotators. In this dissertation, we likewise employ the LLM-as-a-Judge framework, utilizing it as the reward model. This adoption facilitates our transition from RLHF to RLAIF.

2.5 BoN Sampling and the Training-Inference Mismatch

BoN sampling is a well-established and widely adopted inference-time decoding strategy for LLMs (Beirami et al., 2025; Nakano et al., 2022; Ouyang et al., 2022b; Stiennon et al., 2020). At inference time, instead of sampling a single output from the policy π, the model generates N independent completions $\{y_1, \ldots, y_N\} \sim \pi(\cdot \mid x)^{\otimes N}$, where y_i denotes the i-th response and x denotes the given prompt. The final output is the completion with the highest predicted reward, which can be formulated as follows:

$$y_{\mathrm{BoN}}(x) := \arg \max_{y \in \{y_1, \ldots, y_N\}} R(x, y). \tag{2.14}$$

This selection procedure induces a new distribution over completions, which we denote by $T_{\mathrm{BoN}}\pi(y \mid x)$. In this context, we assume that $R(x, y)$ represents the reward function, which yields a deterministic value and follows a continuous distribution. Let $F_R(y)$ denote the cumulative distribution function (CDF) of the reward induced by π. Assuming that ties occur with probability zero, we obtain the following:

$$F_R(y) := \mathbb{P}_{y' \sim \pi(\cdot|x)} \left[R(x, y') \leq R(x, y) \right]. \tag{2.15}$$

This represents the probability that a randomly sampled completion has reward less than or equal to $R(x, y)$. Then, the probability that y is selected as the best among N samples is given by the theory of order statistics:

$$T_{\text{BoN}}\pi(y \mid x) = N \cdot \pi(y \mid x) \cdot F_R(y)^{N-1}. \tag{2.16}$$

This expression captures the probability that y is one of the N generated completions (through $\pi(y \mid x)$), and that the remaining $N - 1$ completions all have reward less than or equal to $R(x, y)$. The factor of N accounts for the N different ways in which y could appear in the N samples and be the maximum. Note that this equation is consistent with the BoN distribution introduced by Balashankar et al. (2025).

The result reveals that $T_{\text{BoN}}\pi(y \mid x)$ is not simply a rescaling of $\pi(y \mid x)$, but a nonlinear transformation that exponentially favors high-reward completions. Specifically, completions in the upper tail of the reward distribution (i.e., those with high $R(x, y)$) are exponentially more likely to be selected, due to the $F_R(y)^{N-1}$ term. Meanwhile, as N increases, this distribution becomes increasingly peaked around the highest-reward completions. In the limit as $N \to \infty$, the distribution collapses to a point mass on the global maximum of the reward:

$$\lim_{N\to\infty} T_{\text{BoN}}\pi(y \mid x) = \mathbb{I}(y = y^*), \quad \text{where } y^* = \arg\max_y R(x, y). \tag{2.17}$$

To support this theoretical derivation, we visualize the transformation from $\pi(y)$ to $T_{\text{BoN}}\pi(y)$ in Figure 2.1. We model the output space y as a continuous scalar variable representing reward, and assume the base distribution $\pi(y) \sim \mathcal{N}(0, 1)$, corresponding to the model's unadjusted reward density. The reward function is deterministic and strictly increasing in y, so that y effectively serves as an index for reward. Under these conditions, we plot equation 2.16 for various values of N. The resulting curves show that even moderate values of N significantly shift probability mass toward the right tail of the distribution, illustrating how BoN acts as a form of selection bias, as stated in Balashankar et al. (2025) as well. It thus violates alignment with the training-time objectives, which are defined with respect to the original model distribution π. The resulting mismatch might hinder the stability of LLM performance.

Our proposed model, MUBPO, addresses this training–inference mismatch by aligning the optimization objectives across both stages through the use of an identical number of completions in each stage and by consistently focusing on the best-performing output. By ensuring that the policy optimization during training matches the objective applied at inference, MUBPO mitigates selection bias, reduces distributional shift, and preserves the consistency of generated outputs across phases. It not only strengthens the robustness of the learned policy, but also improves sample efficiency and yields superior overall performance.

2.6 Marginal Utility

Marginal utility, a foundational concept in microeconomic theory, describes the additional satisfaction or benefit obtained from consuming an incremental unit of a good or service (Menger et al., 1950; von Wieser et al., 1893). In contemporary decision sciences, marginal utility has been abstracted beyond goods and services to any sequential choice problem where an agent evaluates incremental gains relative to a baseline (Keeney & Raiffa, 1993). This generalization extends naturally to our policy gradient framework. Given a prompt x, we generate a set of completions $y_1, y_2, \ldots, y_n$. If we define

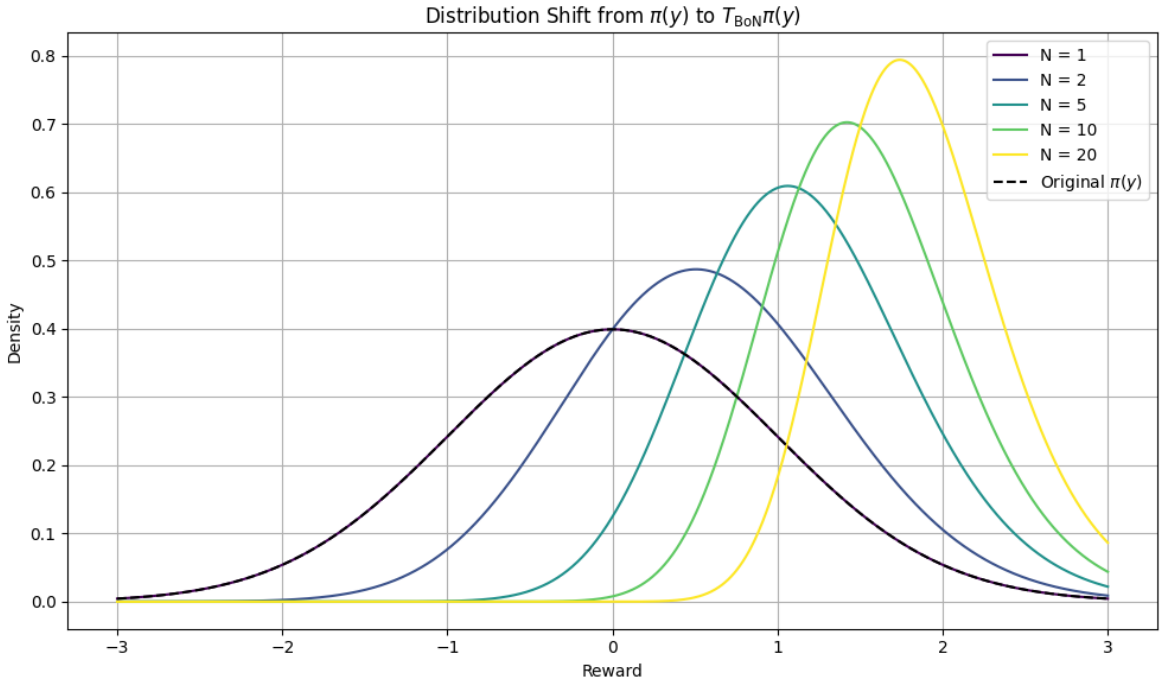


Figure 2.1: Distribution Shift for Best-of-N Sampling

the baseline as the reward for the set excluding y_i, then the marginal utility of the i-th completion can be defined as the reward for the full set minus this baseline. This formulation aligns with economic interpretations of marginal utility as the change in aggregate satisfaction from adding one more unit, and with decision-theoretic treatments where utility is a permutation-invariant function over outcomes (Zaheer et al., 2017). To the best of our knowledge, MUBPO is the first to incorporate the concept of marginal utility into RL algorithms for fine-tuning LLMs.

Chapter 3 Methodology

3.1 MUBPO Framework

We present MUBPO, a novel RL paradigm that utilizes marginal utility theory for policy gradient optimization. In this section, we describe the design of this framework in detail.

Let Ω denote a set of prompts with distribution $\mathcal{D}$ over Ω. For each $x \in \Omega$, we sample n candidate completions independently from the learned policy π_θ, denoted:

$$y_j \sim \pi_\theta(\cdot \mid x), \quad j = 1, \ldots, n \tag{3.1}$$

Define the set of sampled completions as:

$$\mathcal{S} = \{y_1, \ldots, y_n\} \tag{3.2}$$

Let $R(y)$ denote the scalar reward for a response y, and $f : \mathbb{R}^n \to \mathbb{R}$ be a permutation-invariant utility aggregator. The utility function is defined as:

$$u(\mathcal{S}; x) = \mathbb{E}\left[f\left(\{R(y) : y \in \mathcal{S}\}\right)\right] \tag{3.3}$$

Then the general objective can be written in the form:

$$J(\pi) = \mathbb{E}_{x \sim \mathcal{D},\, S \sim \pi_\theta^n(\cdot|x)}\left[u(S; x)\right] - c(\pi) \tag{3.4}$$

where $c : \mathbb{R}^{|\Omega|} \to \mathbb{R}$ is a regularization or penalty term, and $\mathcal{S} \sim \pi^n$ indicates n i.i.d. samples from policy π.

For simplicity, we fix x and write $u(\mathcal{S}) = u(\mathcal{S}; x)$ in the following sections.

Stochastic Reward Evaluation. Since the utility function is an expected value, in each of the m stochastic evaluation rounds (indexed by $t = 1, \ldots, m$), a reward model assigns a scalar score to each sampled completion. Specifically, we define:

$$r_j^{(t)} \triangleq R^{(t)}(y_j) \tag{3.5}$$

where $R^{(t)}$ denotes the reward function evaluated in round t, and $y_j \in \mathcal{S}$ is the j-th candidate. This formulation enables for robust reward estimation through multiple rounds of evaluation. Note that in this case, the reward function is no longer deterministic; otherwise, the expected value would be the same for all rounds, which would be exactly the same as when $m = 1$.

Marginal Utility. Given $\mathcal{S} = \{y_1, \ldots, y_n\}$, the marginal utility of completion y_j can be defined as the difference between the full-set utility and the leave-one-out utility (i.e., the utility without y_j):

$$\begin{aligned} \delta_j &= u(\mathcal{S}) - u\big(\mathcal{S} \setminus \{y_j\}\big) \\ &= \mathbb{E}\left[f(\mathcal{R})\right] - \mathbb{E}\left[f(\mathcal{R}_{-j})\right] \end{aligned} \tag{3.6}$$

where $\mathcal{R} = \{R(y) : y \in \mathcal{S}\}$ and $\mathcal{R}_{-j} = \mathcal{R} \setminus \{R(y_j)\}$.

Applying this definition to an m-round stochastic approximation, indexed by $t = 1, \ldots, m$, we have:

$$\delta_j^{(t)} = f^{(t)}(\mathcal{R}) - f^{(t)}(\mathcal{R}_{-j}) \tag{3.7}$$

and the ensemble estimate is given by:

$$\delta_j = \mathbb{E}_t[\delta_j^{(t)}] = \mathbb{E}_t\left[f^{(t)}(\mathcal{R}) - f^{(t)}(\mathcal{R}_{-j})\right] \approx \frac{1}{m}\sum_{t=1}^{m} \delta_j^{(t)} \tag{3.8}$$

From a statistical standpoint, increasing m reduces the variance and improves the accuracy of the approximation.

3.2 Utility Function Specifications

A variety of utility functions can be employed in MUBPO. This study focuses on three principal and representative examples: (1) the sum utility function, which aligns closely with the standard GRPO framework (DeepSeek-AI et al., 2025); (2) the maximum utility function, which emphasizes the best-performing completion; and (3) the win-rate utility function, which is derived from pairwise comparative outcomes. In addition, several other utility functions, such as the softmax and Lp-norm formulations, also demonstrate strong potential and are worthy of further investigation.

3.2.1 Summation Utility

The summation utility function represents the foundational setting and serves as the canonical form for marginal utility evaluation. It is motivated by a straightforward additive perspective: each sampled completion independently contributes to the total utility based on its reward score. This formulation is ideal for scenarios where absolute performance of each completion matters, without requiring interactions among completions.

The underlying reasoning can be formalized mathematically, as shown below:

$$f\left(\{R(y) : y \in S\}\right) = \sum_{y \in S} R(y) \tag{3.9}$$

Hence, for an m-round approximation, in the t-th round, we have the following:

$$f_{\text{Sum}}^{(t)}(\mathcal{R}) = \sum_{j=1}^{n} r_j^{(t)} \tag{3.10}$$

$$\delta_j^{\text{Sum},(t)} = f^{(t)}(\mathcal{R}) - f^{(t)}(\mathcal{R}_{-j}) = r_j^{(t)} \tag{3.11}$$

When $m = 1$ and $\forall j \in \mathcal{S}$, it follows that:

$$\delta_j^{\text{Sum},(1)} = \delta_j^{\text{Sum}} \equiv r_j^{(1)} \tag{3.12}$$

This utility function is both computationally efficient and conceptually intuitive, and it serves as the standard setting for sequence-level GRPO. However, a key limitation lies in its objective: maximizing the average performance of the group completions. This approach can lead to a mismatch between training and inference, as previously discussed. To address this issue, we propose alternative utility functions that focus more on the performance of the best completion.

3.2.2 Maximum Utility

The maximum utility function is introduced to emphasize and measure best performance within a group. Its core idea is to evaluate the importance of individual completions based on whether they contribute to the highest reward observed in the sample set. This is particularly beneficial in applications where only the best-performing output is of interest, such as BoN setting. So we define as the following:

$$f\left(\{R(y) : y \in \mathcal{S}\}\right) = \max_{y \in \mathcal{S}} R(y) \tag{3.13}$$

Then, for the m-round approximation case in the t-th round, the same logic applies:

$$f_{\text{Max}}^{(t)}(\mathcal{R}) = \max_{j=1,\ldots,n} r_j^{(t)} \tag{3.14}$$

$$\delta_j^{\text{Max},(t)} = \max_{k=1,\ldots,n} r_k^{(t)} - \max_{k=1,\ldots,n,k\neq j} r_k^{(t)} \tag{3.15}$$

This approach yields two possible marginal utility values for each completion:
- $\delta_j^{(t)} > 0$: Sample j uniquely attains the maximum reward in the t-th round.
- $\delta_j^{(t)} = 0$: Sample j either attains a tied maximum reward or receives a non-maximal reward in the t-th round.

This formulation emphasizes completions that are critical to achieving peak quality, thereby facilitating focused optimization on top-tier outputs. However, issues can arise when $\delta_j^{(t)} = 0$ because multiple identical maximum values occur within a group, i.e., when $\max_{k=1,\ldots,n} r_k^{(t)} = \max_{k=1,\ldots,n,\, k\neq j} r_k^{(t)}$. In this case, a slight problem may arise because all marginal utilities are zero, making it difficult for the algorithm to learn anything useful. This emphasizes the importance of introducing the m-round approximation. In practice, conducting multiple scoring rounds and averaging the resulting deltas (i.e., marginal utilities) mitigates this issue by reducing the chance that all marginal utilities within a group are zero, thereby allowing informative signals to be captured for top-tier outputs.

3.2.3 Win Rate Utility*

Note: This method has not been implemented and is excluded from all subsequent experiments in this dissertation.

Inspired by the design of DPO (Rafailov et al., 2023) and NLHF (Munos et al., 2024), we incorporate the idea of pairwise comparison into our optimization framework. Further inspired by

InfAlign (Balashankar et al., 2025), we adopt the win-rate utility function, which aims to capture relative performance instead of absolute reward values. The core objective is to identify completions that outperform a known baseline, which is generally the reference model.

Definition 1 (Win-Rate Utility Function)**.** *Given a set of completions $\mathcal{S}$, let $\mathcal{R} = \{R(y) : y \in \mathcal{S}\}$ be the corresponding set of representations, $\mathcal{R}_{ref}$ be the reference set of representations, and let $\mathbb{I}$ be a indicator function. The win-rate utility function is defined as:*

$$f(\mathcal{R}) = \mathbb{I}\left\{\max \mathcal{R} > \max \mathcal{R}_{ref}\right\} + \frac{1}{2} \cdot \mathbb{I}\left\{\max \mathcal{R} = \max \mathcal{R}_{ref}\right\} \tag{3.16}$$

This definition is identical to the one introduced by InfAlign (Balashankar et al., 2025). Then the t-th round result would be:

$$f^{(t)}_{\text{Comp}}(\mathcal{R}) = \mathbb{I}\{\max \mathcal{R}^{(t)} > \max \mathcal{R}^{(t)}_{\text{ref}}\} + \frac{1}{2} \cdot \mathbb{I}\{\max \mathcal{R}^{(t)} = \max \mathcal{R}^{(t)}_{\text{ref}}\} \tag{3.17}$$

$$\begin{aligned}\delta_j^{\text{Comp},(t)} = &\left[\mathbb{I}\left\{\max \mathcal{R}^{(t)} > \max \mathcal{R}^{(t)}_{\text{ref}}\right\} + \frac{1}{2} \cdot \mathbb{I}\left\{\max \mathcal{R}^{(t)} = \max \mathcal{R}^{(t)}_{\text{ref}}\right\}\right] \\ &- \left[\mathbb{I}\left\{\max \mathcal{R}^{(t)}_{-j} > \max \mathcal{R}^{(t)}_{\text{ref}}\right\} + \frac{1}{2} \cdot \mathbb{I}\left\{\max \mathcal{R}^{(t)}_{-j} = \max \mathcal{R}^{(t)}_{\text{ref}}\right\}\right].\end{aligned} \tag{3.18}$$

where $\mathcal{R}^{(t)}$ denotes the $\mathcal{R}$ in t-th round.

In this setting, the marginal utility values can be divided into three distinct cases:

- $\delta_j^{(t)} = 1$: Sample j is exclusively responsible for outperforming the reference in the t-th round.
- $\delta_j^{(t)} = 0.5$: Sample j is either uniquely responsible for achieving a tie or is pivotal in outperforming the reference, given a tie for the second-highest score between completions in the t-th round.
- $\delta_j^{(t)} = 0$: Sample j is non-essential for achieving comparative advantage in the t-th round in three cases: there are better completions that outperform the reference; there are better completions that tie with the reference; or all completions in the group underperform the reference.

In this setting, an issue analogous to that observed in the maximum utility function case also arises. Specifically, this occurs when all completions generated by the policy receive higher rewards (i.e., scores) than those produced by the reference model. Under such circumstances, all deltas are zero, which may reduce the effectiveness of reinforcement learning. Consequently, the selection of the reference model requires careful consideration.

Overall, this utility function is particularly beneficial in evaluation scenarios where competitive success, measured by win rate, is critical, such as benchmark comparisons or leaderboard-based training.

3.3 Advantage Metric

We use the standardization method for marginal utilities to obtain the advantage value, following the same approach as in sequence-level GRPO (DeepSeek-AI et al., 2025). Let $\mathcal{N} = \{1, \ldots, n\}$ be a finite element set, and $f : 2^{\mathcal{N}} \to \mathbb{R}$ be a set utility function. The *advantage* A_j of an element $j \in \mathcal{N}$ is its

standardized marginal contribution to the utility:

$$A_j = \frac{\delta_j - \mu_\delta}{\sigma_\delta}, \tag{3.19}$$

where:

- δ_j represents the expected marginal utility previously defined,
- $\mu_\delta = \frac{1}{|\mathcal{N}|} \sum_{k \in \mathcal{N}} \delta_k$ is the mean of marginal utilities,
- $\sigma_\delta = \sqrt{\frac{1}{|\mathcal{N}|} \sum_{k \in \mathcal{N}} (\delta_k - \mu_\delta)^2}$ denotes the population standard deviation. Alternatively, the sample standard deviation, which is computed by dividing by $|\mathcal{N}| - 1$ rather than $|\mathcal{N}|$, may also be employed. Furthermore, to prevent numerical instability arising from a zero-valued standard deviation in the denominator, a small constant $\varepsilon = 10^{-8}$ is added to the denominator in subsequent experiments.

This advantage metric transforms raw marginal contributions into a standardized signal for stable optimization. By centring and scaling the marginal utility, we emphasize relative differences in sample quality and mitigate the influence of reward scale variability. This standardization ensures compatibility with policy gradient methods and supports efficient learning dynamics.

3.4 Optimization Framework

The MUBPO constitutes the following objective:

$$\mathcal{J}(\theta) = \underbrace{\mathbb{E}[\min(\rho_t(\theta) A_j, \text{clip}(\rho_t(\theta), 1 \pm \epsilon) A_j)]}_{\text{Surrogate objective}} - \beta \cdot \underbrace{D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})}_{\text{Regularizer}} \tag{3.20}$$

where $\rho_t(\theta) = \frac{\pi_\theta(y_j|x)}{\pi_{\text{old}}(y_j|x)}$ denotes the importance sampling ratio, ϵ is the clipping threshold, and β controls the strength of Kullback–Leibler (KL) divergence regularization. Note that the whole objective is computed at the sequence level rather than the token level, in alignment with C. Zheng et al. (2025) and DeepSeek-AI et al. (2025).

KL Penalty Term. In the LLM setting, the KL penalty term $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ measures how far the current policy π_θ has drifted from the reference policy π_{ref}, acting as a regularizer to keep learning stable and prevent the model from straying too far from its original language ability.

Following the standard practice in RLHF, we apply a KL divergence penalty to constrain the updated policy from deviating excessively from the reference model. Specifically, at each generation step t in a sequence of length T, given a prompt x, we compute the token-level KL divergence between the output distributions of the current policy π and the reference model π_{ref} conditioned on x as:

$$D_{\text{KL},t}(x, y_{<t}) = \sum_{i=1}^{V} \pi_t(i \mid x, y_{<t}) \log \frac{\pi_t(i \mid x, y_{<t})}{\pi_t^{\text{ref}}(i \mid x, y_{<t})} \tag{3.21}$$

where V is the vocabulary size, $y_{<t}$ is all tokens before time step t in the sequence y, and $\pi_t(i)$ is the probability of token i at sequence position t.

However, computing the token-level KL exactly requires too much computation and memory. Therefore, a common choice is to simplify the code by storing only the log-prob and using Monte Carlo

estimation. The most popular unbiased estimator is called k_1 (Schulman, 2020), which is defined as:

$$D_{\mathrm{KL}}(p \,\|\, q) = \log p(x) - \log q(x) \tag{3.22}$$

In our setting, the Monte Carlo estimate can be expressed as follows:

$$D_{\mathrm{KL},t}\left(x^{(b)}, y_{j,<t}^{(b)}\right) = \log \pi_t\left(y_{j,t}^{(b)} \,\middle|\, x^{(b)}, y_{j,<t}^{(b)}\right) - \log \pi_t^{\mathrm{ref}}\left(y_{j,t}^{(b)} \,\middle|\, x^{(b)}, y_{j,<t}^{(b)}\right) \tag{3.23}$$

where b indexes the prompts and j indexes the completions.

GRPO uses the Monte Carlo estimation to calculate token KL as well, but it uses another unbiased estimator k_3 (Schulman, 2020), which is defined as:

$$D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}) = \frac{\pi_{\mathrm{ref}}(y_t \mid x, y_{<t})}{\pi_\theta(y_t \mid x, y_{<t})} - \log \frac{\pi_{\mathrm{ref}}(y_t \mid x, y_{<t})}{\pi_\theta(y_t \mid x, y_{<t})} - 1 \tag{3.24}$$

However, the k_3 estimator has been shown to be incidental in certain cases—when combined with reward maximization, its estimate converges to a policy that deviates from the target optimal policy (Tang & Munos, 2025). Therefore, we retain our choice of k_1.

We aggregate the KL penalty over all tokens in the sequence and then average it across all sequences in the batch:

$$D_{\mathrm{KL,batch}} = \frac{1}{Bn} \sum_{b=1}^{B} \sum_{j=1}^{n} \sum_{t=1}^{T_{b,j}} D_{\mathrm{KL},t}\left(x^{(b)}, y_{j,<t}^{(b)}\right) \tag{3.25}$$

The notation is defined as follows:

- B denotes the batch size;
- $b \in \{1, \ldots, B\}$ indexes individual prompts within the mini-batch;
- n is the number of sequences associated with each prompt;
- $j \in \{1, \ldots, n\}$ indexes sequences or segments for a given prompt;
- $T_{b,j}$ represents the length of the j-th sequence for the b-th prompt;
- $t \in \{1, \ldots, T_{b,j}\}$ indexes the time steps (i.e., positions) within sequence j of prompt b;
- $D_{\mathrm{KL},t}\left(x^{(b)}, y_{j,<t}^{(b)}\right)$ denotes the KL divergence evaluated at time step t, conditioned on the input $x^{(b)}$ and the prefix $y_{j,<t}^{(b)}$, and calculated according to Equation 3.23.

The principal distinction between this sequence-level KL formulation and GRPO's token-level KL computation (Sheng et al., 2024; von Werra et al., 2020) lies in the aggregation method: the former defines the sequence-level divergence as the sum of token-wise KL terms, whereas the latter computes their average over the sequence. The theoretical rationale for this summation is discussed in Appendix II.

Overall, this surrogate objective leverages the PPO-style clipping mechanism to ensure stable updates while incorporating the advantage signal derived from marginal utility. The KL regularization term constrains the updated policy to remain close to a reference distribution, facilitating safe exploration and preventing policy collapse. Together, these components yield a robust optimization framework tailored to RL from marginal utility signals.

3.5 Inference-Time Evaluation

To align the objectives between the training stage and inference-time, the evaluation methodology employed during inference closely resembles the utility function used in the training stage. Specifically, we choose the maximum utility function (see Equation 3.14). The corresponding inference-time evaluation metric for each prompt is defined as follows:

$$\text{Inference-Time Score} = \frac{1}{m} \sum_{t=1}^{m} \max_j r_j^{(t)} \tag{3.26}$$

where m represents the number of rounds, which should be identical to the m used in the training stage, and $\max_j r_j^{(t)}$ denotes the maximum score achieved in the t-th round.

In the standard GRPO setting, the inference procedure simplifies to the special case where $m = 1$, in which case the score reduces to the maximum reward. For other cases which employ the maximum utility function during training, this strategy ensures that the training objective is directly aligned with inference-time behavior. By emphasizing top-performing completions during both training and inference, the model learns to prioritize high-reward outputs, thereby reducing train-test mismatch and improving overall deployment effectiveness.

We evaluate the KL divergence for the validation set using the same procedure employed during training. Specifically, for each prompt, we compute the cumulative KL divergence:

$$D_{\text{KL},i} = \frac{1}{n} \sum_{j=1}^{n} \sum_{t=1}^{T_{i,j}} D_{\text{KL},t}\left(x_i,\, y_{i,j,<t}\right) \tag{3.27}$$

where i is the prompt index, j is the sequence index, $y_{i,j,<t}$ denotes the prefix of the j-th sequence generated from the i-th prompt, up to (but not including) the t-th token, and $T_{i,j}$ is the length of the j-th sequence generated from the i-th prompt.

The overall KL divergence metric for the validation set is then obtained by averaging over all v prompts:

$$D_{\text{KL,val}} = \frac{1}{v} \sum_{i=1}^{v} D_{\text{KL,i}} \tag{3.28}$$

where v denotes the total number of prompts in the dataset.

Chapter 4 Experiments

4.1 Dataset and Reward Model

This study utilizes the `TL;DR` Reddit Summarization dataset, made available through the TRL library (von Werra et al., 2020), which has become a widely used benchmark in summarization research. The dataset contains a collection of Reddit posts in which authors frequently append a "TL;DR" (Too Long; Didn't Read) summary to their submissions. Each example adheres to the prompt–completion schema defined by the TRL framework, comprising a `prompt` field (the full Reddit post) and a `completion` field (the corresponding user-written summary).

In conventional use, such datasets are paired with reward models grounded in lexical overlap metrics, such as ROUGE-L (Lin, 2004). Under this paradigm, model-generated summaries are compared to human-written references, and the similarity score—such as the F1 variant of ROUGE-L—serves as the reward signal during training. Figure 4.1 illustrates the training reward under this ROUGE-based evaluation with `Qwen/Qwen2-0.5B-Instruct` as the pretrain model. Although the average reward increases over training steps, the trajectory exhibits marked fluctuations, reflecting the inherent instability and limited robustness of purely lexical matching approaches.

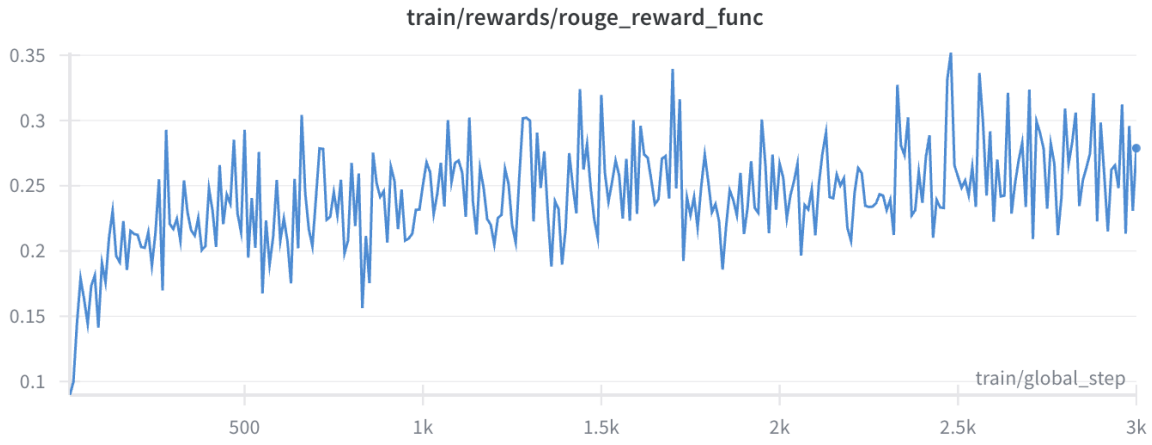


Figure 4.1: Training Reward Curve Using ROUGE-L

In addition, as discussed in Section 2.4, lexical metrics such as ROUGE are inadequate for assessing key qualitative dimensions of summaries, including semantic fidelity, coherence, and informativeness. To overcome these limitations, this work adopts an LLM as the reward model. In this configuration, the `completion` field is no longer required for direct comparison; only the `prompt` is provided to the LLM, which then evaluates and scores the generated summary directly. We denote the `prompt` as `text`. An example of `text` is presented in Table 5.1 of Section 5.1.

4.2 MUBPO Algorithm

The proposed algorithm implements the marginal utility concept introduced in Chapter 3. The complete procedure is presented in Algorithm 1.

Algorithm 1 MUBPO: Marginal Utility-Based Policy Optimization

Require: Prompt set Ω, parameterized policy π_θ, previous policy $\pi_{\theta_{\text{old}}}$, reference policy π_{ref}, reward model $R(\cdot)$, batch size B, completions per prompt n, evaluation rounds m, clipping parameter ϵ, KL coefficient β, utility function f, numerical stabilizer ε, learning rate η

Initialize policy parameters θ; set $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$; load π_{ref}
for each training iteration **do**
  Sample a batch $\{x^{(b)}\}_{b=1}^{B} \subset \Omega$
  for each prompt $x^{(b)}$ **do**
    Generate completions $\mathcal{S}^{(b)} \leftarrow \{\, y_j^{(b)} \sim \pi_\theta(\cdot \mid x^{(b)}) \,\}_{j=1}^{n}$
    Generate reference completions $\mathcal{S}_{\text{ref}}^{(b)} \leftarrow \{\, y_j^{\text{ref},(b)} \sim \pi_{\text{ref}}(\cdot \mid x^{(b)}) \,\}_{j=1}^{n}$
  end for
  for $t = 1$ **to** m **do** ▷ Stochastic reward evaluation passes
    for each $y_j^{(b)} \in \mathcal{S}^{(b)}$ **do**
      $r_j^{(t,b)} \leftarrow R^{(t)}\big(y_j^{(b)}\big)$
    end for
    for each $y_j^{\text{ref},(b)} \in \mathcal{S}_{\text{ref}}^{(b)}$ **do**
      $r_j^{\text{ref},(t,b)} \leftarrow R^{(t)}\big(y_j^{\text{ref},(b)}\big)$
    end for
    Let $\mathcal{R}^{(b)} \leftarrow \{\, r_j^{(t,b)} \,\}_{j=1}^{n}$ and $\mathcal{R}_{-j}^{(b)}$ denote $\mathcal{R}^{(b)}$ with the j-th element removed
    for $j = 1$ **to** n **do**
      $\delta_j^{(t,b)} \leftarrow f^{(t)}\big(\mathcal{R}^{(b)}\big) \;-\; f^{(t)}\big(\mathcal{R}_{-j}^{(b)}\big)$ ▷ Marginal utility
    end for
  end for
  $\delta_j^{(b)} \leftarrow \frac{1}{m}\sum_{t=1}^{m} \delta_j^{(t,b)} \quad \forall j$
  $A_j^{(b)} \leftarrow \dfrac{\delta_j^{(b)} - \mu_{\delta^{(b)}}}{\sigma_{\delta^{(b)}} + \varepsilon}$ ▷ Normalize advantages per b
  $\rho_j^{(b)} \leftarrow \dfrac{\pi_\theta\big(y_j^{(b)} \mid x^{(b)}\big)}{\pi_{\theta_{\text{old}}}\big(y_j^{(b)} \mid x^{(b)}\big)}$ ▷ Importance ratio
  for each completion $y_j^{(b)}$ with $j \in \{1, \ldots, n\}$ **do**
    for each token position $t \in \{1, \ldots, T_{b,j}\}$ **do** ▷ Mask padding as needed
      Calculate Token-level $\mathrm{KL}_t\big(x^{(b)}, y_{j,<t}^{(b)}\big)$
    end for
    $\mathrm{KL}_j^{(b)} \leftarrow \sum_{t=1}^{T_{b,j}} \mathrm{KL}_t\big(x^{(b)}, y_{j,<t}^{(b)}\big)$ ▷ Sum over tokens
  end for
  $\mathrm{KL}_{\text{batch}} \leftarrow \dfrac{1}{B\,n}\sum_{b=1}^{B}\sum_{j=1}^{n} \mathrm{KL}_j^{(b)}$ ▷ Mask padding as needed
  $L_{\text{pg}} \leftarrow -\mathbb{E}\Big[\min\Big(\rho_j^{(b)} A_j^{(b)},\ \mathrm{clip}\big(\rho_j^{(b)},\ 1-\epsilon,\ 1+\epsilon\big) A_j^{(b)}\Big)\Big]$ ▷ PPO-style surrogate
  $L_{\text{total}} \leftarrow L_{\text{pg}} \;+\; \beta \cdot \mathrm{KL}_{\text{batch}}$
  Update $\theta \leftarrow \theta - \eta \nabla_\theta L_{\text{total}}$ with gradient clipping; set $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
  Log metrics and periodically save checkpoints
end for
return π_θ

4.3 Experimental Settings

The policy network in our experiments is based on the `Qwen/Qwen2-0.5B-Instruct` model (Yang et al., 2024), a 0.5-billion-parameter, instruction-tuned transformer derived from Qwen2-0.5B and developed by Alibaba. This model choice aligns with contemporary research practice, including its application in DeepSeek (DeepSeek-AI et al., 2025). It was loaded via HuggingFace's `AutoModelForCausalLM` and served both as the trainable policy and the reference model for KL-regularized updates.

To supply reward signals during both the training and inference stages, we utilize the `Gemini-2.0-Flash` model. This model delivers evaluation scores through an API interface, enabling systematic reward shaping and guiding the policy toward behaviours aligned with the desired objectives. The reward generation procedure operates under fixed temperature and provider settings to ensure consistency across experiments. The temperature is set to 0.1 to avoid fully deterministic scoring (temperature = 0), thereby enabling multiple scoring rounds whose results can be averaged for greater robustness. At the same time, the temperature is kept low to limit variability, as evaluating the quality of a summarization should prioritize consistency and factual accuracy over creativity and diversity.

Role	**Value**
Policy Model	`Qwen/Qwen2-0.5B-Instruct`
Reference Model	`Qwen/Qwen2-0.5B-Instruct`
Reward Model	`models/gemini-2.0-flash`
Reward Provider	`gemini`
Reward Model Temperature	0.1

Table 4.1: Model Configuration

Reproducibility is ensured through the use of fixed random seeds during training. For each seed x, the inference stage uses $x + 1$ to ensure that the evaluation remains consistent while avoiding identical stochastic conditions between training and inference.

Stage	**Seeds Used**
Training	2026, 2027, 2028, 2029, 2030
Inference	2027, 2028, 2029, 2030, 2031

Table 4.2: Random Seed Allocation for Reproducibility

The experiments are executed on a single NVIDIA RTX 6000 GPU. Due to its memory constraints, we adopt a batch size of 1 and limit the total number of optimization steps to 500. Consistent with prevailing findings in large language model research, we expect that increasing batch size, training duration (measured in steps), and model size (i.e., number of parameters) will lead to improved performance, contingent upon the availability of sufficient computational resources.

The optimization process employs the `Adam` optimizer with a learning rate of 1×10^{-6}. KL-regularization is applied with a coefficient of $\beta = 0.1$ and epsilon clipping is set to $\epsilon = 0.05$. Multiple parameter values (e.g., $\beta = 0.01$) were empirically tested, and the final selection reflects an optimal trade-off between convergence stability and learning speed. In this work, β is kept constant rather than adapted dynamically, as the focus is not on regularization strategies; this also ensures consistency across all models considered.

Parameter	Value
Execution Device	`cuda`
CUDA Version	12.9
GPU Model	NVIDIA RTX 6000
Batch Size	1
Total Optimization Steps	500

Table 4.3: Hardware and Execution Settings

Parameter	Value
Optimizer	`Adam`
Learning Rate (η)	1×10^{-6}
KL Coefficient (β)	0.1
Epsilon Clipping (ϵ)	0.05

Table 4.4: Optimizer Hyperparameters

During generation, we sample four completions per prompt with a temperature of 0.5 and a top-p threshold of 0.95. Although a temperature of 0.7 is more commonly adopted in general-purpose language generation, we employ a slightly lower value to better suit the summarization task, where excessive creativity is less desirable and a higher degree of factual precision and consistency is preferred. A top-p threshold of 0.95 refers to the nucleus sampling strategy (Holtzman et al., 2020), in which the model first ranks all possible next tokens by probability and then retains only the smallest subset whose cumulative probability mass reaches at least 95%. This candidate set is subsequently renormalized, and the next token is sampled from it according to the adjusted probabilities, ensuring that higher-probability tokens remain more likely while still allowing limited variability. This approach filters out implausible tokens while preserving a controlled degree of diversity, thereby balancing coherence and flexibility in text generation. Multi-round training uses $m = 3$ rounds, with the baseline GRPO model limited to $m = 1$, with the reason illustrated in Section 3.2.1.

Parameter	Value
Completions per Prompt	4
Temperature	0.5
Top-p	0.95
Max New Tokens	64
Rounds (m)	3 (1 for baseline)

Table 4.5: Sampling and Generation Settings

The training and evaluation data are sourced from the `trl-lib/tldr` dataset. The training set uses the *train* split, while evaluation is performed on the *validation* split.

4.4 Prompt Design

The experimental pipeline employs two distinct prompt types: one for summarization, used by both the policy and reference models (`Qwen2-0.5B-Instruct`), and another for evaluation, directed to the reward model (`Gemini-2.0-Flash`). The detailed prompts can be found in Table 4.7. These prompts

Dataset	Split
Training Set	`trl-lib/tldr` — Train
Test Set	`trl-lib/tldr` — Validation

Table 4.6: Training and Evaluation Datasets

follow the Alpaca-style format (Taori et al., 2023), which served as the original template for instruction fine-tuning. They are applied uniformly across all models during both training and inference to ensure consistency, reproducibility, and fair comparison of outputs. This standardized prompting strategy helps isolate model behaviour from prompt-induced variance, thereby improving the reliability of performance evaluation.

Task Type	Prompt Design	Explanation
Summarization	`Summarize this Reddit post briefly:` `{p}`	{p} denotes the full text of the Reddit post.

Scoring	`Rate the overall quality of the summary compared to the Reddit post.`
	`Provide a SINGLE number between 0 (worst) and 100 (best).`
	`Respond with ONLY the number, no explanations.`
	
	`ORIGINAL TEXT:`
	`{text}`
	
	`SUMMARY:`
	`{summ}`
	
	`SCORE (0--100):`
		{text} denotes the original post, {summ} denotes the generated summary.

Table 4.7: Prompt Designs for Summarization and Scoring Tasks

4.5 Model Specifications

In our experiments, we evaluate three configurations of our framework, designed to isolate the effects of key design choices:

1. **Baseline: Standard GRPO** (*standard* ■; displayed in blue in subsequent figures): Employs the summation-based utility function described in Section 3.2.1, with $m = 1$ round and $n = 4$ completions per prompt. This corresponds to the conventional sequence-level GRPO setting.
2. **Maximum-Utility,** $m = 3$ (*max_m3* ■; displayed in dark grey in subsequent figures): Implements the maximum-utility function (Section 3.2.2) with $m = 3$ rounds and $n = 4$ completions per prompt.
3. **Maximum-Utility,** $m = 3$**,** $n = 8$ (*max_m3_n8* ■; displayed in green in subsequent figures): Extends *max_m3* by increasing completions per prompt from $n = 4$ to $n = 8$.

Comparisons are conducted in two stages and can be interpreted as an ablation study:

1. *standard* vs. *max_m3* isolates the effect of replacing the summation-based utility function (GRPO)

with the maximum-based formulation (MUBPO) under otherwise comparable conditions, thereby assessing whether MUBPO yields improvements over GRPO. Theoretically, the answer is affirmative, as MUBPO optimizes based on the best observed performance rather than the average performance, as discussed in Section 2.3.2.

2. *max_m3* vs. *max_m3_n8* isolates the effect of increasing the number of completions per prompt while keeping the utility function and the number of rounds fixed, thereby evaluating whether additional completions enhance performance. Theoretically, the answer is affirmative, as a larger number of completions is expected to yield a higher maximum utility value, as shown in Section 2.5.

Chapter 5 Results and Interpretation

5.1 Sanity Check: Sample Completions

Before conducting large-scale evaluations, a qualitative sanity check was performed to verify that the completions follows general writing rules and scoring framework produces scores and rankings consistent with human judgement. At step 499, a single input example was processed by three model configurations: *standard*, *max_m3*, and *max_m3_n8*. The input text, along with the highest-scoring completion from each model, is shown in Table 5.1.

The completions were scored by the same reward model `gemini-2.0-flash`. The *standard* output scored 85, the *max_m3* output scored 90, and the *max_m3_n8* output scored 95. The score progression aligns with a qualitative review of the outputs.

The *standard* output, which scored 85, delivers the most abstract and generalized summary. It captures the overall theme of the post: doing something to make someone proud, yet still disappointing them, while also seeking other experiences or perspectives. However, it omits all of the concrete details from the original post. This makes it useful for high-level topic classification but less effective for fully understanding the situation.

The *max_m3* output, scoring 90, is richer in detail and more narrative in tone. It includes key facts, while also conveying the tension between the author's passion for Entomology and the grandmother's preference for a medical career. This output feels more personal and engaging, but it contains a subtle factual distortion: it implies that the author chose to pursue medical school, whereas in the prompt they chose Entomology instead. Despite this, it retains much of the emotional nuance missing from the *standard* output.

Finally, the *max_m3_n8* output earns the highest score, 95, for its strong factual accuracy and completeness. It clearly states the academic milestones, the graduate school plans in Entomology, and the grandmother's ongoing disappointment. Its precision and coverage of key facts make it the most reliable of the three completions.

Overall, this sanity check demonstrates that the evaluation framework (i.e., the reward model) correctly ranks higher-quality, factually accurate, and more complete outputs above those that are vague or contain inaccuracies. The results provide preliminary evidence that the scoring system reflects human judgment and is suitable for subsequent large-scale analysis. In addition, since Table 5.1 is based on step 499, the final step of model training, the three outputs' scores reflect the overall performance of the three models after training. Notably, the ranking of the three outputs' scores aligns with the ranking of the three models' final inference performance.

5.2 Training Stage: *standard* vs. *max_m3*

During training, extensive information is recorded using Weights & Biases (Biewald, 2020) to ensure that the process is proceeding as expected. Among these records, several plots are particularly informative.

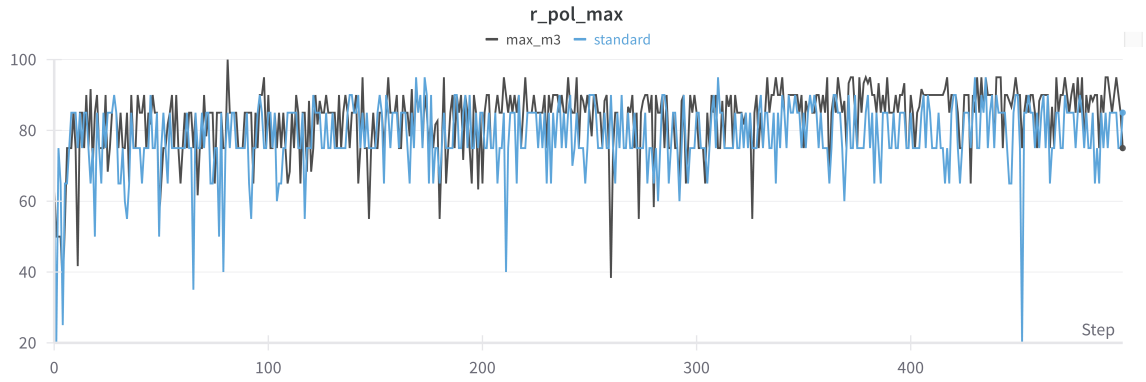
Component	Content
Data Input ({`text`})	`SUBREDDIT: r/AskReddit` `TITLE: What is something that you've done to make someone proud, but still ended up a disappointment to them?` `POST:` `Granted I'm still very proud of myself and that's what matters but, I've been working really hard in college. I'm graduating on time, with 2 degrees (Entomology and Chemistry), and with distinction. I'm going to grad school for a masters in Entomology then further to a PhD. I told my grandma about this and she told me that I'm going amount to nothing... she wants me to be a medical doctor even though I told her how happy I am in Entomology. I want to make her proud but she is still disappointed in me. Has anyone else had a similar experience?` `TL;DR:`
standard **Output**	`Reddit post asking for insights on people who've done something to make someone proud, but ended up disappointed, asking for other experiences or perspectives on how to handle such situations.`
max_m3 **Output**	`Reddit user asks about an experience where the individual got to graduate on time and with distinction, but their grandma still expresses disappointment in their decision to pursue medical school, despite their passion for Entomology. They're seeking advice on how to make their grandma proud.`
max_m3_n8 **Output**	`Reddit user asks if there is anything that they have done to make someone proud and still end up disappointing them. They have graduated on time with their degrees and are going to grad school for a masters in Entomology. They want to make their grandma proud but she is still disappointed in them.`

Table 5.1: Model Comparison with Input Text and Generated Summaries

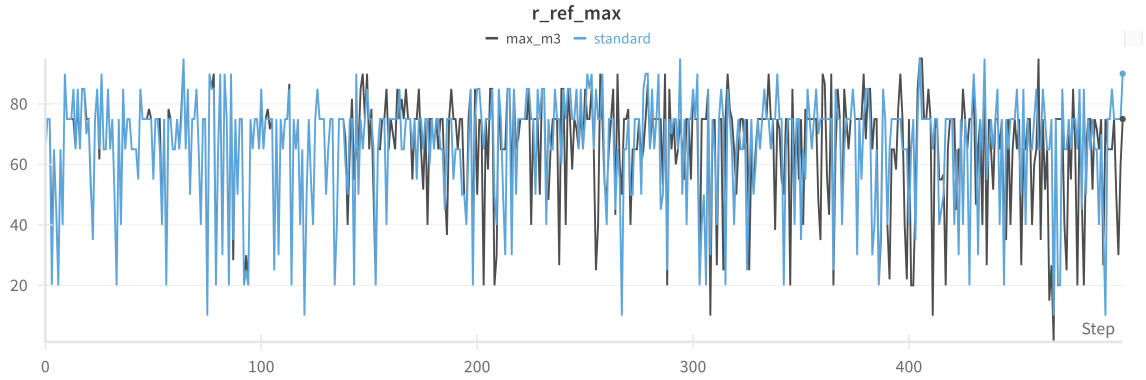
Model Score vs. Step. The first plot illustrates the relationship between the maximum score within each group of completions generated by the policy, denoted as $r_{\text{pol_max}}$, and the training step, as shown

in Figure 5.1a. This can be compared with Figure 5.1b, which shows the corresponding relationship for the maximum score in completions generated by the reference policy, denoted as $r_{\text{ref_max}}$. To enable a fair comparison, the same random seed is used for both the *standard* and *max_m3* models. This also ensures that the two plots in Figure 5.1b align closely.

In Figure 5.1a, both models initially exhibit some variability with occasional dips but quickly stabilize within the higher reward range of approximately 70–90. They demonstrate more stable and better performance than the reference model, which shows no clear upward trend and fluctuates broadly between approximately 20 and 90. The *max_m3* model achieves slightly higher scores and experiences fewer severe drops than the *standard* model, indicating marginal improvements in performance and stability.



(a) Max Score vs Step (Policy)



(b) Max Score vs Step (Reference)

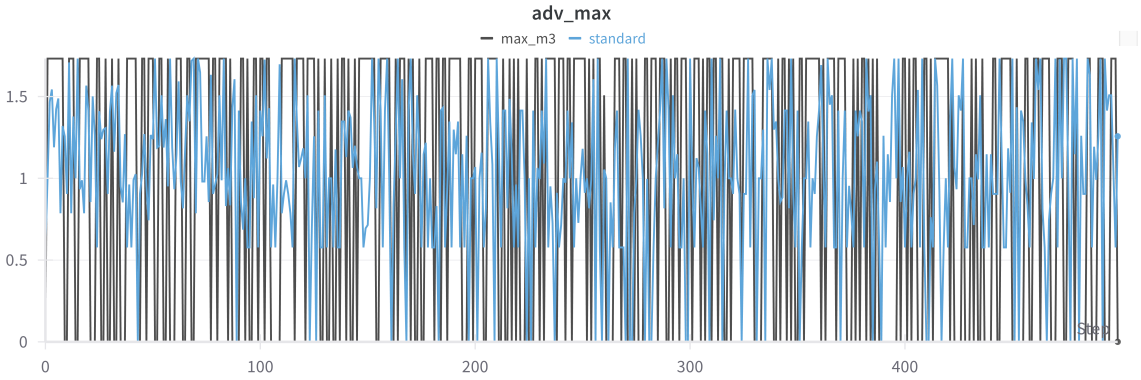
Figure 5.1: Max Score vs. Step for Current Policy and Reference Policy (*standard* vs. *max_m3*)

Advantage vs. Step. As outlined in Section 3, the advantage function is derived from the reward signal. Figure 5.2 presents the maximum and minimum advantage values observed for each input prompt during the training process. Compared to the standard GRPO model (*standard*), our proposed model (*max_m3*) exhibits both higher maximum and lower minimum advantage values.

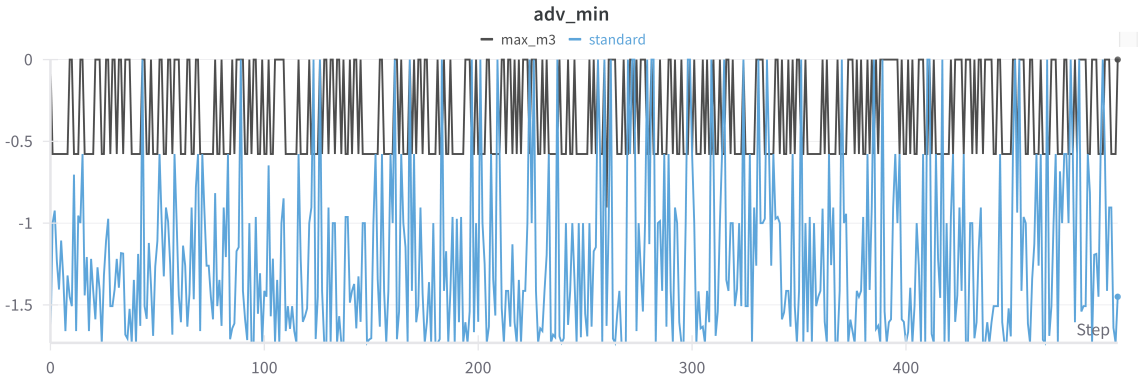
Additionally, it is worth noting that under the *standard* case, both the maximum and minimum advantage values are dispersed across a range of outcomes, whereas for *max_m3* model, the maximum and minimum advantage values occur only at several values. In particular, the maximum advantage fluctuates between 0 and $\sqrt{3} \approx 1.732$, while the minimum advantage fluctuates between 0 and

$-\frac{1}{\sqrt{3}} \approx -0.577$. This is consistent with expectations: for a reward vector of the form $[0, 0, 0, a]$ with a single positive entry $a > 0$, the mean is $\mu = a/4$ and the population variance is $\sigma^2 = 3a^2/16$, yielding $\sigma = a\sqrt{3}/4$. Standardization then gives $z = (0-\mu)/\sigma = -1/\sqrt{3}$ for the three zero entries, and $z = (a-\mu)/\sigma = \sqrt{3}$ for the single positive entry. Thus, any such vector yields standardized advantages of three values at $-1/\sqrt{3}$ and one value at $\sqrt{3}$. If we switch from population standard deviation to sample standard deviation, the values change slightly, but the shape of the plots would not change.

Taken together, in the *standard* model, the advantage values are relatively symmetric, whereas in the *max_m3* model, the distribution is more skewed. For *max_m3*, since the maximum advantage value reaches a relatively high magnitude of 1.732, this may indicate a stronger preference for the corresponding action, as it provides a more pronounced gradient signal and is likely to facilitate more effective exploitation.



(a) Maximum Advantage vs Step



(b) Minimum Advantage vs Step

Figure 5.2: Maximum and Minimum Advantage vs Step (*standard* vs. *max_m3*)

Loss vs. Step. The Figure 5.3 illustrates the evolution of the KL divergence (sequence-level) across training steps for *max_m3* and *standard*. In this scenario, the KL divergence reflects how the learned distribution diverges from the reference distribution over time, thus serving as an indicator of model regularization.

Both configurations demonstrate an increasing trend in KL divergence as training progresses, indicating a growing deviation of the model's learned distribution from the reference distribution. Notably, the *max_m3* configuration shows more pronounced fluctuations and generally higher KL

values between steps 150 and 400. The *standard* configuration maintains lower and more stable KL values during early and mid training rounds, but exhibits a rapid increase at a later stage.

Overall, while *max_m3* model may permit more aggressive exploration earlier in training, it reaches to a distribution closer to the reference distribution by the end. Meanwhile, the *standard* configuration appears more conservative initially but ultimately allows greater divergence. From a regularization perspective, the lower final KL in *max_m3* could indicate stronger constraints on the learned distribution, potentially enhancing generalization. These patterns highlight the importance of monitoring KL dynamics across the full training trajectory rather than relying solely on final KL values.

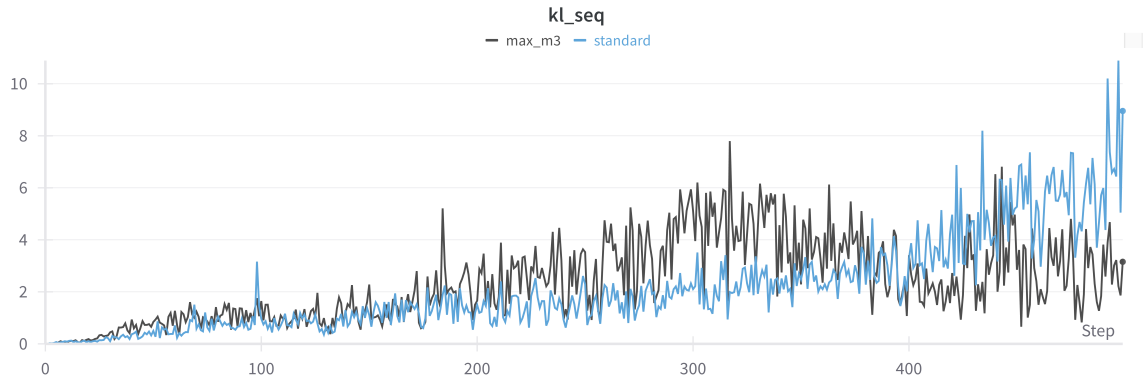


Figure 5.3: KL Divergence vs Step (*standard* vs. *max_m3*)

5.3 Inference Stage: *standard* vs. *max_m3*

Model Scores: *max_m3* vs. *standard*. Figure 5.4a and Table 5.2a summarize the scores obtained across five seeds, demonstrating that both the *standard* and *max_m3* methods achieve high average performance, with mean scores of approximately 83.23 and 84.21, respectively. Table 5.2a further shows that *max_m3* outperforms *standard* on every seed. However, the 95% confidence intervals for these means overlap, indicating that the observed difference in performance is not statistically significant given the current sample size. Moreover, a closer examination of the confidence interval widths indicates that *max_m3* shows slightly less variability in performance than *standard*, suggesting a more consistent behaviour across the evaluation dataset. In practical terms, this implies that while both methods achieve comparably high performance, *max_m3* model may provide a marginally better outcome, with improved score stability and higher score values.

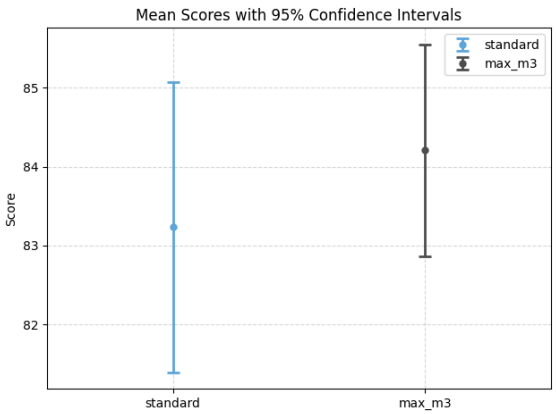
***standard* Model**	***max_m3* Model**
85.87	85.99
82.35	83.89
82.50	83.18
82.70	84.33
82.75	83.64

(a) Score

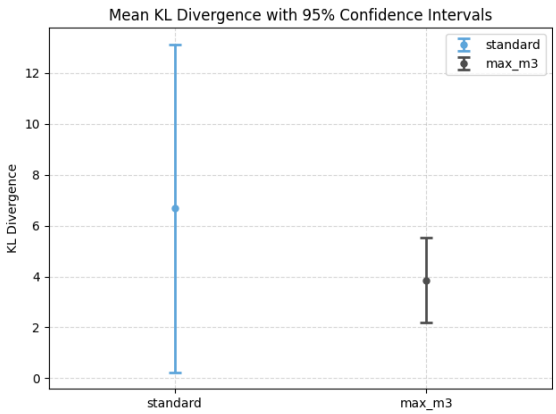
***standard* Model**	***max_m3* Model**
5.3954	5.3903
6.2598	3.4504
2.7327	2.0006
3.3993	3.5279
15.6178	4.9162

(b) KL Divergence

Table 5.2: Performance Comparison Between *standard* and *max_m3* Models



(a) Mean Score with 95% Confidence Intervals



(b) Mean KL with 95% Confidence Intervals

Figure 5.4: Mean Score and Mean KL Divergence with 95% Confidence Intervals (*standard* vs. *max_m3*)

KL Divergences: *max_m3* vs. *standard*. The analysis of KL divergence, shown in Figure 5.4b and Table 5.2b, highlights a pronounced difference between the two methods. The *standard* method demonstrates a considerably higher mean KL divergence, approximately 6.68, coupled with a notably wide 95% confidence interval. This broad interval is largely driven by an extremely large value (≈ 15.6), which reflects an occasional, substantial deviation in policy updates. By comparison, *max_m3* maintains a lower mean KL divergence of approximately 3.86, with a much narrower confidence interval. Additionally, note that *max_m3*'s KL divergence value is smaller than *standard*'s on almost every seed. Therefore, *max_m3* produces more controlled and consistent policy updates, indicating more stable and reliable training dynamics.

Taken together, these results suggest that *max_m3* is capable of achieving slightly better performance than *standard*, with markedly improved stability in terms of policy divergence. From an technical perspective, lower and more stable KL divergence implies that *max_m3* mitigates the risk of destabilizing policy updates, which is particularly desirable in RL process where uncontrolled divergence can lead to performance degradation. Consequently, while the performance scores of *max_m3* over *standard* appear to be modest, its superior stability represents a significant practical advantage for the robust fine-tuning stage.

Model Score vs. KL Divergence. The plotted results in Figure 5.5 present a comparative analysis between the *max_m3* and *standard* policy configurations, evaluated across five training checkpoints (steps 100, 200, 300, 400, and 500). The x-axis represents the KL divergence, which quantifies the extent to which the updated policy diverges from the reference distribution, averaged across random seeds. The y-axis represents the average score, a performance metric obtained by averaging across multiple random seeds. Each data point on the plot corresponds to a specific training step, with the associated KL-score pair illustrating the trade-off between policy performance and divergence from the original model (i.e., the reference or pretrain model).

It is worth noting that the figure may also be interpreted as representing a fundamental trade-off in RL, namely the exploration–exploitation dilemma (Berger-Tal et al., 2014; Sutton & Barto, 2018), wherein the average score can be regarded as a measure of exploitation, while the average KL divergence

functions as a proxy for exploration.

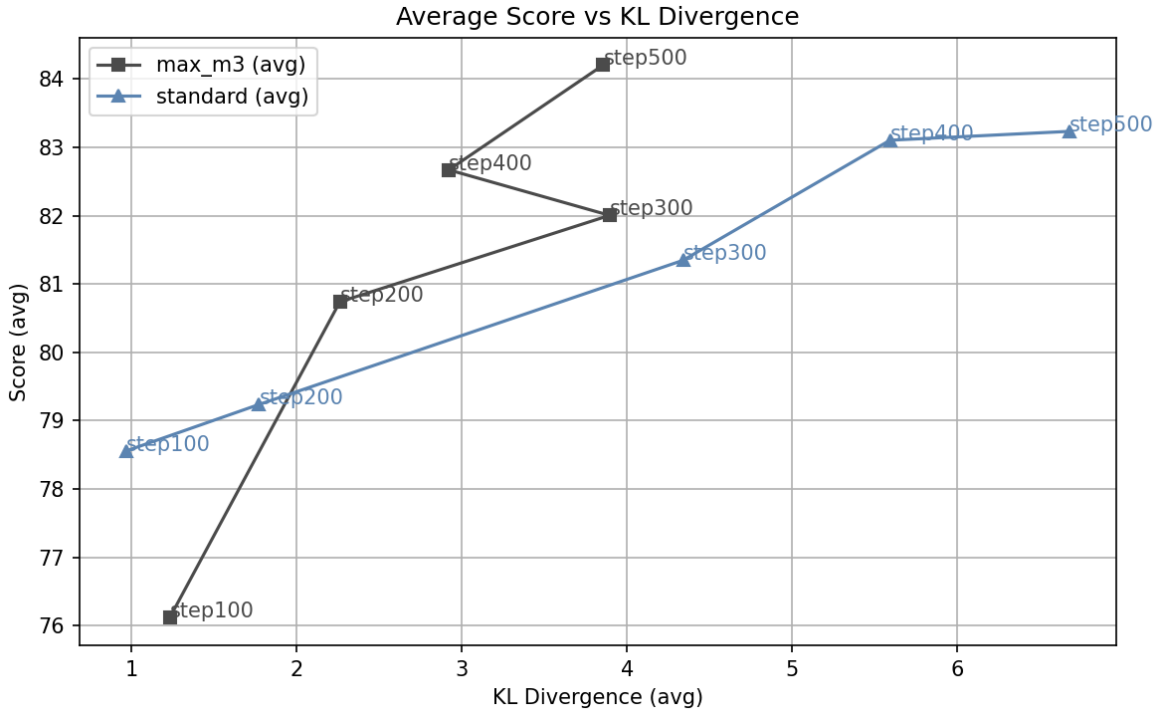


Figure 5.5: Score vs KL Divergence (*standard* vs. *max_m3*)

In general, we would like the score (i.e., exploitation) to be neither too high nor too low: if it is too high, the policy may overfit to short-term rewards at the expense of generalization, while if it is too low, the policy may fail to capture useful patterns. Similarly, we would like the KL divergence (i.e., exploration) to be balanced, avoiding the risks of both over-exploration and under-exploration. Overall, we would expect the curve in Figure 5.5 to be approximately monotonic increasing in shape.

The *max_m3* model generally demonstrates an improvement in score over the course of training, beginning at approximately 76.1 at step 100 and reaching 84.2 by step 500. However, its trajectory is not strictly monotonic. Between steps 300 and 400, the score increases only slightly from 82.0 to 82.7, with a decrease in KL divergence from 3.90 to 2.92. This suggests that the model may have temporarily moved closer to the reference policy with a small gain in performance. Between steps 400 and 500, both the score and the KL divergence rise noticeably again, reaching 84.2 and 3.86, respectively. This suggests a renewed policy update that departs further from the reference and leads to improved performance. Such non-monotonic shape reflects a dynamic learning process in which the policy occasionally stabilizes before achieving additional gains. Furthermore, increasing the batch size is expected to smooth the learning curve, reducing fluctuations and producing a trajectory that is more monotonic.

In contrast, the *standard* model, visualized with muted blue triangles, initially exhibits higher scores than *max_m3* during early training steps, starting at approximately 78.6 at step 100. However, the KL divergence for this configuration grows much more rapidly, surpassing 6.6 by step 500. While the score improves steadily to 83.2, its growth remains very slow and plateaus after step 400. This pattern suggests that the *standard* model is relatively inefficient, as it incurs substantially higher divergence without achieving correspondingly better performance.

Overall, these results highlight a clear trade-off between policy divergence and score: achieving a higher score generally requires a higher KL divergence. The *max_m3* configuration achieves superior final performance with lower divergence, indicating a more favourable balance between stability and reward optimization.

5.4 Training Stage: *max_m3* vs. *max_m3_n8*

Model Score vs. Step. Consistent with the previous model comparison, the evolution of $r_{\text{pol_max}}$ during training is presented in Figure 5.6 for *max_m3* and *max_m3_n8*. In both cases, the values exhibit a rapid increase over the first approximately 30 steps, after which they both stabilize and oscillate between 75 and 95. Beyond roughly step 220, the maximum policy reward in the group completions for *max_m3_n8* consistently exceeds that of *max_m3*. This outcome is both expected and desirable, as the maximum value observed in a group of eight samples is, in expectation, higher than that observed in a group of four drawn from the same distribution. The result is therefore consistent with, and supportive of, the claim stated in Section 2.5.

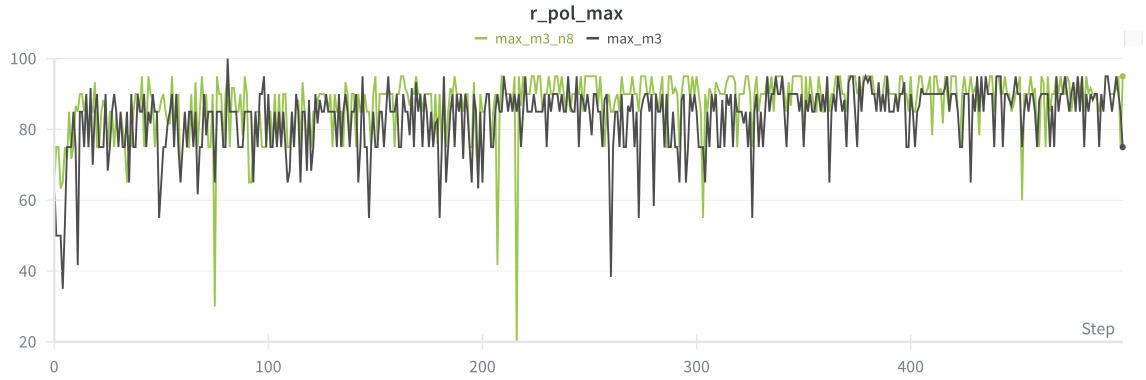


Figure 5.6: Max Score vs Step for Current Policy (*max_m3* vs. *max_m3_n8*)

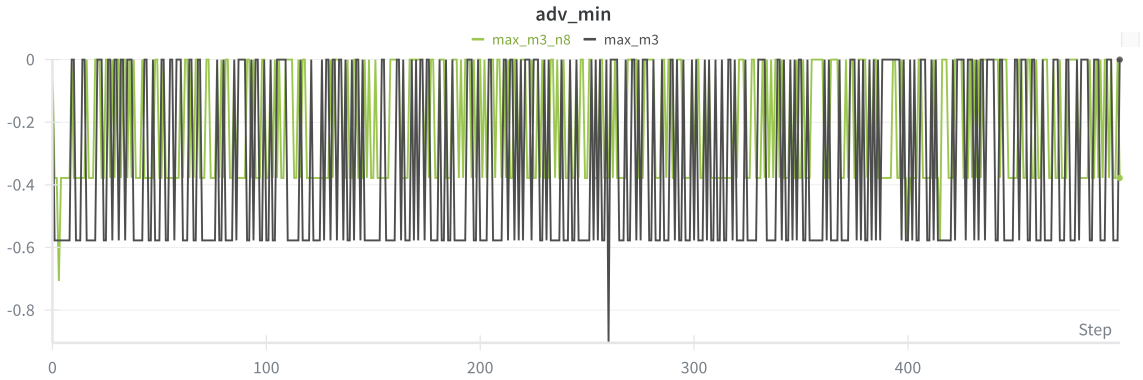
Advantage vs. Step. Increasing the number of completions from four to eight, while holding all other settings constant, modifies the range of observed advantages: the maximum within each set increases, whereas the minimum decreases (see Figure 5.7).

Similar to the case for $n = 4$ in *max_m3* discussed above, consider a reward vector of the form $[0, 0, 0, 0, 0, 0, 0, a]$ with a single positive entry $a > 0$. The mean is $\mu = a/8$, and the population variance is $\sigma^2 = 7a^2/64$, giving $\sigma = a\sqrt{7}/8$. Standardization then yields $z = (0-\mu)/\sigma = -1/\sqrt{7} \approx -0.378$ for the seven zero entries, and $z = (a-\mu)/\sigma = \sqrt{7} \approx 2.646$ for the single positive entry. Hence, any such vector produces standardized advantages of seven values at $-1/\sqrt{7}$ and one value at $\sqrt{7}$, confirming the same structural asymmetry as in the $n = 4$ case.

Overall, the $n = 8$ case in *max_m3_n8* yields a larger maximum advantage, which produces a stronger gradient signal toward exploitation. In general, this is more beneficial for RL when the environment is stable and the best action can be reliably identified. In our experimental setting, the feedback signal is reliable and consistent; hence, it is preferable to emphasize the best completion more strongly, thereby accelerating exploitation.



(a) Maximum Advantage vs Step



(b) Minimum Advantage vs Step

Figure 5.7: Maximum and Minimum Advantage vs Step (*max_m3* vs. *max_m3_n8*)

Loss vs. Step. Figure 5.8 presents the sequence-level KL divergence (KL_{seq}) over the course of training for the *max_m3* and *max_m3_n8* models. Both models start with zero KL divergence, which then increase steadily during the initial training phase (approximately the first 100 steps). Beyond this point, the *max_m3* line increases rapidly, exhibiting pronounced fluctuations with frequent sharp peaks exceeding values of 4–6, which indicate episodes of substantial divergence from the reference distribution. In contrast, *max_m3_n8* maintains its steady growth, with a comparatively lower KL divergence until the end of training, along with a far narrower fluctuation range and far fewer extreme spikes.

This pattern suggests that *max_m3_n8* not only maintains closer alignment to the reference distribution, but also exhibits reduced variance in policy updates. The lower and more stable KL divergence indicates more controlled exploration and less instability in learning, which are generally associated with better generalization and reduced risk of overfitting. Consequently, these results support the view that *max_m3_n8* is a more stable and consistently better-performing model compared to *max_m3*.

5.5 Inference Stage: *max_m3* vs. *max_m3_n8*

Model Scores: *max_m3* vs. *max_m3_n8*. Figure 5.9a presents the mean task scores, along with 95% confidence intervals across five seeds, for *max_m3* and *max_m3_n8* models. The results indicate that *max_m3_n8* achieves an approximately 3 unit higher mean score compared to *max_m3*, with only

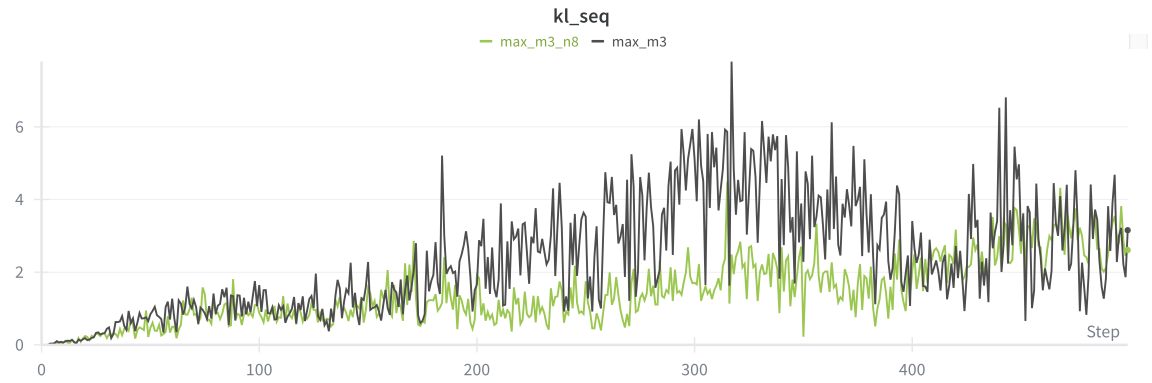
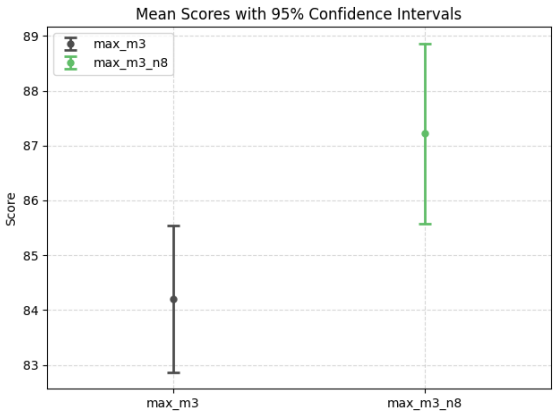


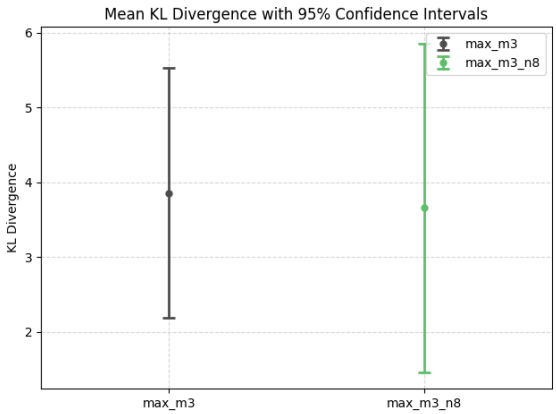
Figure 5.8: KL Divergence vs Step (*max_m3* vs. *max_m3_n8*)

minimal overlap observed between their respective 95% confidence intervals. This separation suggests that the observed performance gain is likely to be statistically significant at the 95% confidence level. Notably, the widths of the confidence intervals are comparable across the two settings, implying that increasing the number of completions per prompt from $n = 4$ to $n = 8$ does not adversely affect the stability of task scores. In addition, for every seed, the score of *max_m3_n8* exceeds that of *max_m3*. These findings all indicate that *max_m3_n8* delivers a tangible performance improvement, although it incurs little additional computational cost.

KL Divergences: *max_m3* vs. *max_m3_n8*. Figure 5.9b reports the mean KL divergence for the same configurations. Here, *max_m3_n8* exhibits a slightly lower mean KL divergence relative to *max_m3*. However, the 95% confidence intervals for both methods are wide and substantially overlapping. Moreover, *max_m3_n8* shows a slightly larger confidence interval, reflecting greater variability in KL divergence outcomes. This increased variability may suggest reduced stability when a larger number of completions is used.



(a) Mean Score with 95% Confidence Intervals



(b) Mean KL with 95% Confidence Intervals

Figure 5.9: Mean Score and Mean KL Divergence with 95% Confidence Intervals (*max_m3* vs. *max_m3_n8*)

Overall, doubling n appears beneficial for improving evaluation scores, with a stable and consistent KL divergence. Its effect on KL divergence, however, remains slightly less conclusive than that

of *max_m3* and may involve a trade-off between potential marginal gains and increased variance. Nevertheless, the gain in score is relatively large, and the observed increase in KL variance is relatively small, so this is unlikely to be a major concern in practical applications.

Chapter 6 Conclusion

We propose the MUBPO framework, a novel RL fine-tuning paradigm designed to directly address the training–inference misalignment problem in LLMs, which exists in a number of policy optimization algorithms, including GRPO. In the conventional token-level GRPO setting, rewards are assigned to completions, while optimization is based on tokens. Sequence-level GRPO eliminates this reward–optimization mismatch, but optimizes the average performance of a set of completions rather than the best completion within the set. MUBPO bridges the gap between the objectives optimized during training and the selection procedures employed at inference time, particularly in the case of BoN sampling, by focusing on the best completion through the use of marginal utility.

This work contributes a unifying formulation that extends GRPO as a special case, offering a generalized utility-based optimization framework capable of accommodating diverse utility functions. The marginal utility formulation provides a principled means of quantifying the incremental contribution of each completion in a candidate set, which not only enhances credit assignment but also improves stability in preference-based fine-tuning. This framework enriches the design of RLHF fine-tuning by leveraging economic utility theory in RL, a direction that has not previously been explored.

The experimental evaluation on the `TL;DR` Reddit Summarization dataset provides quantitative evidence of the effectiveness of MUBPO. It consistently outperformed the GRPO baselines in the summarization task, achieving a score improvement of approximately +1% and a reduction in KL divergence of about 4 per sequence. These results indicate that MUBPO achieves better alignment with target behaviour without sacrificing training stability. Moreover, the *max_m3_n8* configuration, which increases the number of completions per prompt from 4 to 8, achieved a further performance gain of approximately +3% score relative to *max_m3*, with KL divergence level unchanged. This improvement suggests that richer sampling promotes task performance at inference-time.

Beyond numerical performance improvements, this framework has a greater significance for general RL policy optimization methods. First, extending MUBPO beyond maximum utility formulations to alternative objectives, such as the win rate utility function, enables the algorithm to align better with the inference goal such as the win rate between the trained model and a baseline model. Second, MUBPO is not limited to LLM applications. It could be used in other RL fields, such as robotics, recommender systems, and interactive decision-making environments.

However, MUBPO has several limitations. From a RL perspective, our focus on selecting the best completion within a group emphasizes exploitation at the expense of exploration. Reaching a balance between these two remains an open question and requires further investigation. From an experimental perspective, the study was restricted to a single dataset and task type, leaving open questions about the generalizability of the approach to other domains such as code generation, dialogue, or multi-turn reasoning. Additionally, all experiments were conducted with only five random seeds. Using more seeds may be necessary to obtain more conclusive results, as the confidence interval would be expected to be narrower.

In summary, this work makes an extensive contribution to the field of LLM alignment by introducing a principled, generalizable, and empirically validated framework for preference-based optimization. By inserting inference-time targets directly into the training process, MUBPO mitigates the misalignment

problem in the current RLHF fine-tuning stage and improves overall performance. These advances lay a solid foundation for developing more reliable, efficient, and human-aligned language models for both research and real-world applications.

Bibliography

Bahdanau, D., Brakel, P., Xu, K., Goyal, A., Lowe, R., Pineau, J., Courville, A., & Bengio, Y. (2017). An actor-critic algorithm for sequence prediction. https://arxiv.org/abs/1607.07086

Balashankar, A., Sun, Z., Berant, J., Eisenstein, J., Collins, M., Hutter, A., Lee, J., Nagpal, C., Prost, F., Sinha, A., Suresh, A. T., & Beirami, A. (2025). Infalign: Inference-aware language model alignment. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=hInfvt7c4p

Banerjee, S., & Lavie, A. (2005, June). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In J. Goldstein, A. Lavie, C.-Y. Lin, & C. Voss (Eds.), *Proceedings of the ACL workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization* (pp. 65–72). Association for Computational Linguistics. https://aclanthology.org/W05-0909/

Barrett, L., & Narayanan, S. (2008). Learning all optimal policies with multiple criteria. *Proceedings of the 25th International Conference on Machine Learning*, 41–47. https://doi.org/10.1145/1390156.1390162

Beirami, A., Agarwal, A., Berant, J., D'Amour, A., Eisenstein, J., Nagpal, C., & Suresh, A. T. (2025). Theoretical guarantees on the best-of-n alignment policy. https://arxiv.org/abs/2401.01879

Berger-Tal, O., Nathan, J., Meron, E., & Saltz, D. (2014). The exploration-exploitation dilemma: A multidisciplinary framework. *PLOS ONE*, *9*(4), e95693. https://doi.org/10.1371/journal.pone.0095693

Biewald, L. (2020). Machine learning experiment tracking with weights and biases [Software available from wandb.com]. https://www.wandb.com/

Böhm, F., Gao, Y., Meyer, C. M., Shapira, O., Dagan, I., & Gurevych, I. (2019, November). Better rewards yield better summaries: Learning to summarise without references. In K. Inui, J. Jiang, V. Ng, & X. Wan (Eds.), *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (emnlp-ijcnlp)* (pp. 3110–3120). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1307

Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 30). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf

The claude 3 model family: Opus, sonnet, haiku. (2024). https://api.semanticscholar.org/CorpusID:268232499

DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., . . . Zhang, Z. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. https://arxiv.org/abs/2501.12948

Hancock, B., Bordes, A., Mazare, P.-E., & Weston, J. (2019, July). Learning from dialogue after deployment: Feed yourself, chatbot! In A. Korhonen, D. Traum, & L. Màrquez (Eds.), *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 3667–3684). Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-1358

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. https://openreview.net/forum?id=rygGQyrFvH

Ji, J., Qiu, T., Chen, B., Zhang, B., Lou, H., Wang, K., Duan, Y., He, Z., Vierling, L., Hong, D., Zhou, J., Zhang, Z., Zeng, F., Dai, J., Pan, X., Ng, K. Y., O'Gara, A., Xu, H., Tse, B., . . . Gao, W. (2025). Ai alignment: A comprehensive survey. https://arxiv.org/abs/2310.19852

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12), 1–38. https://doi.org/10.1145/3571730

Keeney, R. L., & Raiffa, H. (1993). *Decisions with multiple objectives: Preferences and value trade-offs*. Cambridge University Press.

Konda, V., & Tsitsiklis, J. (1999). Actor-critic algorithms. In S. Solla, T. Leen, & K. Müller (Eds.), *Advances in neural information processing systems* (Vol. 12). MIT Press. https://proceedings.neurips.cc/paper_files/paper/1999/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf

Kreutzer, J., Khadivi, S., Matusov, E., & Riezler, S. (2018, June). Can neural machine translation be improved with user feedback? In S. Bangalore, J. Chu-Carroll, & Y. Li (Eds.), *Proceedings of the 2018 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 3 (industry papers)* (pp. 92–105). Association for Computational Linguistics. https://doi.org/10.18653/v1/N18-3012

Lee, H., Phatale, S., Mansoor, H., Mesnard, T., Ferret, J., Lu, K. R., Bishop, C., Hall, E., Carbune, V., Rastogi, A., & Prakash, S. (2024, 21–27 Jul). RLAIF vs. RLHF: Scaling reinforcement learning from human feedback with AI feedback. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, & F. Berkenkamp (Eds.), *Proceedings of the 41st international conference on machine learning* (pp. 26874–26901, Vol. 235). PMLR. https://proceedings.mlr.press/v235/lee24t.html

Li, D., Sun, R., Huang, Y., Zhong, M., Jiang, B., Han, J., Zhang, X., Wang, W., & huan liu. (2025). Preference leakage: A contamination problem in LLM-as-a-judge. *Data in Generative Models - The Bad, the Ugly, and the Greats*. https://openreview.net/forum?id=gW6NT2IuME

Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. *Text Summarization Branches Out*, 74–81. https://aclanthology.org/W04-1013/

Liu, Z., Wang, P., Xu, R., Ma, S., Ruan, C., Li, P., Liu, Y., & Wu, Y. (2025). Inference-time scaling for generalist reward modeling. https://arxiv.org/abs/2504.02495

Menger, C., Dingwall, J., Hoselitz, B. F., & Knight, F. H. (1950). *Principles of economics : First, general part*. Free Press.

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016, 20–22 Jun). Asynchronous methods for deep reinforcement learning. In M. F. Balcan

& K. Q. Weinberger (Eds.), *Proceedings of the 33rd international conference on machine learning* (pp. 1928–1937, Vol. 48). PMLR. https://proceedings.mlr.press/v48/mniha16.html

Munos, R., Valko, M., Calandriello, D., Gheshlaghi Azar, M., Rowland, M., Guo, Z. D., Tang, Y., Geist, M., Mesnard, T., Fiegel, C., Michi, A., Selvi, M., Girgin, S., Momchev, N., Bachem, O., Mankowitz, D. J., Precup, D., & Piot, B. (2024, 21–27 Jul). Nash learning from human feedback. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, & F. Berkenkamp (Eds.), *Proceedings of the 41st international conference on machine learning* (pp. 36743–36768, Vol. 235). PMLR. https://proceedings.mlr.press/v235/munos24a.html

Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., Jiang, X., Cobbe, K., Eloundou, T., Krueger, G., Button, K., Knight, M., Chess, B., & Schulman, J. (2022). Webgpt: Browser-assisted question-answering with human feedback. https://arxiv.org/abs/2112.09332

Ng, A., & updated by Tengyu Ma. (2022). Cs229 lecture notes, spring 2022 [Accessed: 2025-08-14]. https://cs229.stanford.edu/lectures-spring2022/main_notes.pdf

Novikova, J., Dušek, O., Cercas Curry, A., & Rieser, V. (2017, September). Why we need new evaluation metrics for NLG. In M. Palmer, R. Hwa, & S. Riedel (Eds.), *Proceedings of the 2017 conference on empirical methods in natural language processing* (pp. 2241–2252). Association for Computational Linguistics. https://doi.org/10.18653/v1/D17-1238

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., . . . Zoph, B. (2024). Gpt-4 technical report. https://arxiv.org/abs/2303.08774

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., & Lowe, R. (2022a). Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems* (pp. 27730–27744, Vol. 35). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., & Lowe, R. (2022b). Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems* (pp. 27730–27744, Vol. 35). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf

Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). Bleu: A method for automatic evaluation of machine translation. *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, 311–318. https://doi.org/10.3115/1073083.1073135

Provakar, M. M. (2024). Evaluating the text summarization efficiency of large language models. *2024 2nd International Conference on Information and Communication Technology (ICICT)*, 6–10. https://doi.org/10.1109/ICICT64387.2024.10839646

Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in neural information processing systems* (pp. 53728–53741, Vol. 36). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2023/file/a85b405ed65c6477a4fe8302b5e06ce7-Paper-Conference.pdf

Schulman, J. (2020, March). Approximating kl divergence [Accessed: 2025-08-16]. http://joschu.net/blog/kl-approx.html

Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015, July). Trust region policy optimization. In F. Bach & D. Blei (Eds.), *Proceedings of the 32nd international conference on machine learning* (pp. 1889–1897, Vol. 37). PMLR. https://proceedings.mlr.press/v37/schulman15.html

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., & Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. *CoRR*, *abs/1506.02438*. https://api.semanticscholar.org/CorpusID:3075448

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. https://arxiv.org/abs/1707.06347

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models. https://arxiv.org/abs/2402.03300

Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., & Wu, C. (2024). Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*.

Stiennon, N., Ouyang, L., Wu, J., Ziegler, D., Lowe, R., Voss, C., Radford, A., Amodei, D., & Christiano, P. F. (2020). Learning to summarize with human feedback. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, & H. Lin (Eds.), *Advances in neural information processing systems* (pp. 3008–3021, Vol. 33). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2020/file/1f89885d556929e98d3ef9b86448f951-Paper.pdf

Sutton, R., & Barto, A. (2018). *Reinforcement learning, second edition: An introduction*. MIT Press. https://books.google.co.uk/books?id=sWV0DwAAQBAJ

Tang, Y., & Munos, R. (2025). On a few pitfalls in kl divergence gradient estimation for rl. https://arxiv.org/abs/2506.09477

Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., & Hashimoto, T. B. (2023). Stanford alpaca: An instruction-following llama model. https://crfm.stanford.edu/2023/03/13/alpaca.html

Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., Silver, D., Johnson, M., Antonoglou, I., Schrittwieser, J., Glaese, A., Chen, J., Pitler, E., Lillicrap, T., Lazaridou, A., ... Vinyals, O. (2025). Gemini: A family of highly capable multimodal models. https://arxiv.org/abs/2312.11805

Vedantam, R., Zitnick, C. L., & Parikh, D. (2015). Cider: Consensus-based image description evaluation. *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4566–4575. https://doi.org/10.1109/CVPR.2015.7299087

von Werra, L., Belkada, Y., Tunstall, L., Beeching, E., Thrush, T., Lambert, N., Huang, S., Rasul, K., & Gallouédec, Q. (2020). Trl: Transformer reinforcement learning.

von Wieser, F., Malloch, C., & Smart, W. (1893). *Natural value*. Macmillan; Company. https://books.google.co.uk/books?id=ACpBAAAAIAAJ

Wang, H., Xiong, W., Xie, T., Zhao, H., & Zhang, T. (2024, November). Interpretable preferences via multi-objective reward modeling and mixture-of-experts. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Findings of the association for computational linguistics: Emnlp 2024* (pp. 10582–10592). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-emnlp.620

Wang, Z., Bapst, V., Heess, N., Mnih, V., Munos, R., Kavukcuoglu, K., & de Freitas, N. (2017). Sample efficient actor-critic with experience replay. *International Conference on Learning Representations*. https://openreview.net/forum?id=HyM25Mqel

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, *8*(3-4), 229–256.

Wu, J., Ouyang, L., Ziegler, D. M., Stiennon, N., Lowe, R., Leike, J., & Christiano, P. (2021). Recursively summarizing books with human feedback. https://arxiv.org/abs/2109.10862

Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Li, C., Liu, D., Huang, F., Dong, G., Wei, H., Lin, H., Tang, J., Wang, J., Yang, J., Tu, J., Zhang, J., Ma, J., . . . Fan, Z. (2024). Qwen2 technical report. https://arxiv.org/abs/2407.10671

Ye, J., Wang, Y., Huang, Y., Chen, D., Zhang, Q., Moniz, N., Gao, T., Geyer, W., Huang, C., Chen, P.-Y., Chawla, N. V., & Zhang, X. (2025). Justice or prejudice? quantifying biases in LLM-as-a-judge. *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=3GTtZFiajM

Yi, S., Goel, R., Khatri, C., Cervone, A., Chung, T., Hedayatnia, B., Venkatesh, A., Gabriel, R., & Hakkani-Tur, D. (2019, October). Towards coherent and engaging spoken dialog response generation using automatic conversation evaluators. In K. van Deemter, C. Lin, & H. Takamura (Eds.), *Proceedings of the 12th international conference on natural language generation* (pp. 65–75). Association for Computational Linguistics. https://doi.org/10.18653/v1/W19-8608

Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., & Smola, A. J. (2017). Deep sets. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 30). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/f22e4747da1aa27e363d86d40ff442fe-Paper.pdf

Zheng, C., Liu, S., Li, M., Chen, X.-H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., & Lin, J. (2025). Group sequence policy optimization. https://arxiv.org/abs/2507.18071

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in neural information processing systems* (pp. 46595–46623, Vol. 36). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf

Zhou, Z., Liu, J., Shao, J., Yue, X., Yang, C., Ouyang, W., & Qiao, Y. (2024, August). Beyond one-preference-fits-all alignment: Multi-objective direct preference optimization. In L.-W. Ku, A. Martins, & V. Srikumar (Eds.), *Findings of the association for computational linguistics: Acl 2024* (pp. 10586–10613). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.630

Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., Christiano, P., & Irving, G. (2020). Fine-tuning language models from human preferences. https://arxiv.org/abs/1909.08593

Appendix

I Policy Gradient Derivation

Consider the RL objective function

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t r(s_t, a_t)\right], \tag{1}$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory sampled from the policy π_θ and $\gamma \in [0, 1)$ is the discount factor.

The probability of a trajectory τ under policy π_θ is given by

$$p_\theta(\tau) = \rho(s_0)\prod_{t=0}^{\infty}\pi_\theta(a_t \mid s_t)P(s_{t+1} \mid s_t, a_t), \tag{2}$$

where $\rho(s_0)$ is the initial state distribution and $P(s_{t+1} \mid s_t, a_t)$ is the environment transition probability. Using this, the objective can be rewritten as

$$J(\theta) = \int p_\theta(\tau)\left[\sum_{t=0}^{\infty}\gamma^t r(s_t, a_t)\right] d\tau. \tag{3}$$

Then the gradient of $J(\theta)$ is:

$$\nabla_\theta J(\theta) = \nabla_\theta \int p_\theta(\tau)R(\tau)\, d\tau, \tag{4}$$

$$= \int \nabla_\theta p_\theta(\tau)R(\tau)\, d\tau, \tag{5}$$

where we define the return of a trajectory as

$$R(\tau) = \sum_{t=0}^{\infty}\gamma^t r(s_t, a_t). \tag{6}$$

Applying the log-derivative trick,

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\nabla_\theta \log p_\theta(\tau), \tag{7}$$

we obtain

$$\nabla_\theta J(\theta) = \int p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)R(\tau)\, d\tau = \mathbb{E}_{\tau\sim\pi_\theta}\left[\nabla_\theta \log p_\theta(\tau)R(\tau)\right]. \tag{8}$$

The log of the trajectory probability is

$$\log p_\theta(\tau) = \log \rho(s_0) + \sum_{t=0}^{\infty}\log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{\infty}\log P(s_{t+1} \mid s_t, a_t). \tag{9}$$

Since only the policy depends on θ, the gradient simplifies to

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t). \tag{10}$$

Substituting this into our gradient expression yields the policy gradient theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \left(\sum_{k=0}^{\infty} \gamma^k r(s_k, a_k) \right) \right]. \tag{11}$$

It is often convenient to express this using the return from time step t,

$$G_t = \sum_{k=t}^{\infty} \gamma^{k-t} r(s_k, a_k), \tag{12}$$

so that

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t \right]. \tag{13}$$

□

II Decomposition of Sequence-Level KL Divergence

Given an input prompt x and a generated response sequence $y = (y_1, y_2, \ldots, y_T)$, the total KL divergence between the two policies over the entire sequence can be expressed as:

$$D_{\mathrm{KL}}(\pi_\theta(y|x) \,\|\, \pi_{\mathrm{ref}}(y|x)) = \sum_y \pi_\theta(y|x) \log \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)} \tag{14}$$

$$= \mathbb{E}_{y\sim\pi_\theta(\cdot|x)} \left[\log \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)} \right] \tag{15}$$

Since both π_θ and π_{ref} are autoregressive language models, the sequence-level likelihoods decompose via the chain rule:

$$\pi_\theta(y|x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid y_{<t}, x), \quad \pi_{\mathrm{ref}}(y|x) = \prod_{t=1}^{T} \pi_{\mathrm{ref}}(y_t \mid y_{<t}, x) \tag{16}$$

Thus, the KL divergence can be rewritten as a sum of token-level log-probability differences:

$$\log \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)} = \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid y_{<t}, x)}{\pi_{\mathrm{ref}}(y_t \mid y_{<t}, x)} \tag{17}$$

This implies that the total KL divergence over the sequence can be computed as the sum of token-wise KL divergences, where each token's KL is computed between the categorical distributions of $\pi_\theta(\cdot \mid y_{<t}, x)$ and $\pi_{\mathrm{ref}}(\cdot \mid y_{<t}, x)$:

$$D_{\mathrm{KL}}(\pi_\theta(y|x) \,\|\, \pi_{\mathrm{ref}}(y|x)) = \sum_{t=1}^{T} D_{\mathrm{KL}} \left(\pi_\theta(\cdot \mid y_{<t}, x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid y_{<t}, x) \right) \tag{18}$$

□